

RAPPORT DE STAGE DE FIN D'ÉTUDES

En vue de l'obtention du

Diplôme d'Ingénieur en Informatique et Réseaux

Option : Méthodes Informatiques Appliquées à la Gestion des Entreprises (MIAGE)

&

**Master Méthodes Informatiques Appliquées à la Gestion des
Entreprises (MIAGE)**

Parcours : Intelligence Artificielle Appliquée (IA2)

Conception et mise en œuvre d'un pipeline automatisé de contrôle de la qualité des données : Cas des données CSP et KYC

Réalisé par :

Saad BENDAHOU

Encadrants Professionnels :

Mr. Mohamed Agouzzal · CIH Bank

Mr. Marouan Naoui · CIH Bank

Encadrant Pédagogique :

Mr. Marouan Skiti

Organisme d'accueil :



Année universitaire 2025 – 2026

Dédicaces

Je dédie ce modeste travail :

À mes chers parents,

Aucun mot, aucune dédicace ne saurait exprimer mon amour, mon respect et ma profonde gratitude pour vos sacrifices, votre soutien inconditionnel et vos prières qui m'ont toujours accompagné tout au long de mon parcours universitaire. Que Dieu vous préserve et vous procure santé et longue vie.

À mes frères et sœurs,

Pour votre présence constante, vos encouragements et votre affection fraternelle.

À toute ma famille, mes proches et mes amis,

Qui ont toujours cru en moi et m'ont soutenu durant les moments les plus intenses de mes études.

Merci d'être toujours là pour moi.

Remerciements

Avant tout, je tiens à exprimer ma profonde gratitude à Dieu le Tout-Puissant pour m'avoir donné la force, la patience et la persévérance nécessaires pour mener à bien ce travail.

Je tiens à exprimer mes sincères remerciements à mon école, l'École Marocaine des Sciences de l'Ingénieur (EMSI), qui m'accompagne depuis le début de mon cursus de 5 ans, ainsi qu'à l'Université Côte d'Azur de Nice, pour m'avoir assuré une excellente formation académique et des bases solides pour entamer ma vie professionnelle.

J'adresse mes remerciements les plus chaleureux à mon encadrant pédagogique, **Mr. Marouan Skiti**, pour ses conseils méthodologiques avisés, sa disponibilité continue et la bienveillance dont il a fait preuve tout au long du suivi de ce projet de fin d'études.

Mes remerciements les plus vifs s'adressent à l'institution de mon stage, **CIH BANK**, pour m'avoir ouvert ses portes et permis de réaliser ce travail dans un environnement professionnel stimulant et formateur.

J'exprime toute ma reconnaissance à mes encadrants de stage chez CIH Bank, **Mr. Mohamed Agouzzal** (CDO - Chef du Data Office) et **Mr. Marouan Naoui** (Data Quality Analyst / Spécialiste Data Management & Gouvernance), pour leur accueil chaleureux, leur confiance, leur précieux encadrement et leur soutien technique rigoureux durant toute la durée de mon projet. Leurs directives éclairées m'ont grandement aidé à surmonter les difficultés techniques et à concevoir ce pipeline.

Enfin, je remercie l'ensemble des collaborateurs de CIH Bank et les membres de l'équipe Data Office pour leur esprit d'équipe, leur soutien et les échanges enrichissants que nous avons partagés. Que tous ceux qui ont contribué de près ou de loin à la réussite de ce projet trouvent ici l'expression de ma profonde gratitude.

Résumé

Ce projet de fin d'études porte sur la conception, le développement et l'industrialisation d'un pipeline automatisé de contrôle de la qualité des données (Data Quality) au sein du Data Office de CIH BANK. Dans un écosystème bancaire de plus en plus orienté vers le pilotage par la donnée, l'objectif principal consiste à automatiser la validation et la surveillance des données du référentiel Tiers (incluant les informations KYC et les catégories socio-professionnelles CSP) afin de garantir la stricte conformité réglementaire (notamment la loi 09-08 de la CNDP au Maroc, le RGPD européen, et la norme BCBS 239) tout en éliminant les processus chronophages de remédiation manuelle.

La solution développée repose sur une architecture Big Data robuste et s'articule autour d'un pipeline complet : l'ingestion massive des données depuis les systèmes opérants (Oracle NOVA) via Apache Sqoop, l'exécution distribuée de règles de qualité métier (complétude, validité, intégrité, fraîcheur) grâce à l'intégration de l'outil Soda Core avec Apache Spark, et le stockage des résultats dans des tables Hive au format ORC. La couche sémantique est assurée par le moteur de virtualisation Dremio, qui alimente directement un tableau de bord interactif conçu sous Tableau Desktop. Ce dernier permet aux Data Stewards de superviser les indicateurs de performance (KPIs) de qualité, de visualiser les taux de conformité globaux et d'identifier instantanément les anomalies critiques. L'ensemble de ce flux de traitement est orchestré de manière quotidienne et automatisée par Apache Airflow.

Les résultats obtenus démontrent que l'industrialisation des contrôles de qualité à la source améliore significativement la fiabilité du Data Lake, renforce la gouvernance des données, et assure aux décideurs métiers un accès rapide à des informations fiabilisées et sécurisées. Cette architecture offre à la banque une solution évolutive, performante et adaptée aux standards d'ingénierie modernes.

Mots-clés : Qualité des Données (Data Quality), Big Data, Soda Core, Apache Spark, Hadoop, Hive, Dremio, Tableau, Apache Airflow, KYC, CSP, Gouvernance des Données, CIH BANK.

Abstract

This final-year engineering project focuses on the design, development, and industrialization of an automated Data Quality pipeline within the Data Office of CIH BANK. In a banking ecosystem increasingly driven by data, the primary objective is to automate the validation and monitoring of Third-Party domain data (including KYC information and socio-professional categories CSP) to ensure strict regulatory compliance (including the Moroccan CNDP Law 09-08, the European GDPR, and BCBS 239) while eliminating time-consuming manual remediation processes.

The developed solution relies on a robust Big Data architecture and is built around a comprehensive pipeline : mass data ingestion from operational systems (Oracle NOVA) via Apache Sqoop, distributed execution of business quality rules (completeness, validity, integrity, freshness) through the integration of Soda Core with Apache Spark, and storage of the results in Hive tables using the ORC format. The semantic layer is managed by the Dremio virtualization engine, which directly feeds an interactive dashboard built with Tableau Desktop. This dashboard enables Data Stewards to monitor quality Key Performance Indicators (KPIs), track overall compliance rates, and instantly identify critical anomalies. The entire workflow is orchestrated on a daily, automated basis by Apache Airflow.

The results demonstrate that industrializing quality checks at the source significantly improves the reliability of the Data Lake, strengthens data governance, and ensures that business decision-makers have rapid access to verified and secure information. This architecture provides the bank with a scalable, high-performance solution that meets modern engineering standards.

Keywords : Data Quality, Big Data, Soda Core, Apache Spark, Hadoop, Hive, Dremio, Tableau, Apache Airflow, KYC, CSP, Data Governance, CIH BANK.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	x
Introduction Générale	1
Chapitre 1 : Présentation du cadre de projet	3
1.1 Présentation de l'organisme d'accueil	4
1.1.1 Missions de CIH BANK	4
1.1.2 Secteurs d'activité	4
1.1.3 Filiales	5
1.1.4 Actionnariat	5
1.1.5 Chiffres clés	5
1.1.6 Organigramme	6
1.1.6.1 Structure hiérarchique de CIH BANK	6
1.1.7 Data Office	7
1.2 Étude de l'existant	8
1.2.1 Description de l'existant	8
1.2.2 Critique de l'existant	10
1.2.3 Solution proposée	10
1.3 Choix du modèle de développement	11
1.4 Planning prévisionnel	11
Chapitre 2 : Analyse des Besoins et Spécification	13
2.1 Besoins fonctionnels	14
2.1.1 Automatisation et planification des contrôles	14
2.1.2 Configuration déclarative des règles de qualité	14
2.1.3 Historisation et traçabilité des indicateurs	15
2.1.4 Restitution visuelle et aide à la décision	15
2.2 Besoins non fonctionnels	15
2.2.1 Performance	15
2.2.2 Disponibilité	15
2.2.3 Maintenabilité	16
2.2.4 Sécurité	16
2.2.5 Conformité Réglementaire et Normative	16
2.3 Modélisation des cas d'utilisation	16

2.3.1	Acteurs du système	16
2.3.2	Diagramme global des cas d'utilisation	17
2.3.3	Description textuelle des cas d'utilisation principaux	17
2.3.3.1	Cas d'utilisation 01 : Définir et formaliser les règles de qualité	18
2.3.3.2	Cas d'utilisation 02 : Exécuter le scan de qualité (automatique)	18
2.3.3.3	Cas d'utilisation 03 : Consulter le dashboard et remédier aux anomalies	19
Chapitre 3 : Conception de la Solution		20
3.1	Modélisation dynamique	21
3.1.1	Diagramme de séquences	21
3.1.2	Diagramme d'activités	22
3.2	Modélisation statique	22
3.2.1	Modélisation de la table de faits	22
3.2.2	Modèle relationnel	23
3.2.3	Dictionnaire de données	24
3.2.4	Catalogue des règles de qualité	24
3.2.4.1	Dimension Complétude	25
3.2.4.2	Dimension Conformité	25
3.2.4.3	Dimension Cohérence	26
3.2.4.4	Dimension Unicité	26
3.2.4.5	Dimension Actualité et Pipeline	26
3.3	Architecture de l'application	27
3.3.1	Architecture logicielle (diagramme de composants)	27
3.3.1.1	Conception du moteur DQ — Soda Core	28
3.3.1.2	Conception de l'orchestration Airflow	29
3.3.1.3	Conception du dashboard Tableau	29
3.3.2	Architecture matérielle et diagramme de déploiement	30
3.3.2.1	Orchestration technique et protocole de sécurité Kerberos	32
3.3.3	Technologies de l'écosystème utilisées	32
Chapitre 4 : Réalisation de la Solution		34
4.1	Environnement technique de développement	35
4.1.1	Environnement Hadoop et accès distant	35
4.1.2	Installation hors-ligne de Soda Core	36
4.2	Implémentation du moteur DQ — Soda Core et Spark	36
4.2.1	Arborescence et packaging du projet DQ (<code>dq_tiers</code>)	36
4.2.2	Principe de fonctionnement de Soda Core avec Spark	37
4.2.3	Fichier de configuration YAML (<code>checks_tiers.yml</code> — exemple)	38
4.2.4	Traduction des règles SodaCL en Spark SQL	39
4.2.5	Script d'exécution principal (<code>dq_tiers_endpoint.py</code>)	40
4.3	Création des tables Hive physiques de persistance	43
4.4	Création du DAG Airflow indépendant (<code>dq_tiers_daily00</code>)	44
4.5	Exposition et virtualisation sous Dremio	45
4.6	Réalisation du dashboard Tableau	46
4.6.1	Connexion Tableau à Dremio et traitement analytique	46
4.6.2	Onglet 1 — Vue d'ensemble (Dashboard Global)	47
4.6.3	Onglet 2 — Heatmap Qualité	48
4.6.4	Onglet 3 — Top Anomalies	49

Conclusion Générale et Perspectives	50
Bibliographie et Webographie	52

Table des figures

1	Chiffres clés de CIH BANK (31/12/2024)	5
2	Organigramme de CIH BANK	7
3	Architecture matérielle Big Data	9
4	Planning prévisionnel du projet	12
5	Diagramme global des cas d'utilisation de la solution DQ Tiers	17
6	Diagramme de séquences du pipeline de contrôle et restitution DQ	21
7	Diagramme d'activités du workflow de remédiation DQ	22
8	Modélisation de la grande table de faits <code>fact_tiers_dq_consolidated</code> et ses 3 tables sources	23
9	Diagramme de composants logiciels de la solution DQ	28
10	Diagramme de déploiement UML 2.5 de la solution Data Quality	31
11	DBeaver	32
12	MobaXterm	32
13	Apache Hive	32
14	Apache Spark	33
15	Soda Core	33
16	Apache Airflow	33
17	Dremio	33
18	Tableau	33
19	GitLab	33
20	Interface MobaXterm utilisée pour l'accès SSH au nœud Edge du cluster Cloudera	35
21	Lignage des données dans Dremio montrant la création de la vue sémantique à partir des 3 tables Hive	46
22	Interface de connexion entre Tableau Desktop et le serveur Dremio	47
23	Dashboard Tableau — Vue d'ensemble de la qualité des données	47
24	Dashboard Tableau — Heatmap Qualité par colonne et par date	48
25	Dashboard Tableau — Top anomalies et colonnes en échec	49

Liste des tableaux

2	Informations générales sur le groupe CIH BANK [10]	4
3	Répartition du capital de CIH BANK [4]	5
4	Acteurs du système de Data Quality	17
5	Description du cas d'utilisation 01	18
6	Description du cas d'utilisation 02	18
7	Description du cas d'utilisation 03	19
8	Dictionnaire simplifié des tables de faits de Qualité des Données	24
9	Dimension Complétude — Données Tiers (Référentiel Tiers)	25
10	Dimension Conformité — Données Tiers (Référentiel Tiers)	25
11	Dimension Cohérence — Données Tiers (Référentiel Tiers)	26
12	Dimension Unicité — Données Tiers (Référentiel Tiers)	26
13	Dimension Actualité et Pipeline — Flux Batch	27
14	Versions des composants de l'environnement technique	36

Liste des abréviations

Abréviation	Signification
ANRF	Autorité Nationale du Renseignement Financier
API	Application Programming Interface
BCBS	Basel Committee on Banking Supervision
BI	Business Intelligence
CDO	Chief Data Officer
CIH	Crédit Immobilier et Hôtelier
CNDP	Commission Nationale de contrôle de la Protection des Données à caractère personnel
CNSS	Caisse Nationale de Sécurité Sociale
CSP	Catégories Socio-Professionnelles
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DQ	Data Quality
DWH	Data Warehouse
ETL	Extract, Transform, Load
HDFS	Hadoop Distributed File System
ISO	Organisation Internationale de Normalisation
KPI	Key Performance Indicator
KYC	Know Your Customer
LLM	Large Language Model
ORC	Optimized Row Columnar
PDM	Python Dependency Manager
RGPD	Règlement Général sur la Protection des Données
SQL	Structured Query Language
YAML	Yet Another Markup Language

Introduction Générale

L'évolution rapide des technologies numériques et l'augmentation exponentielle du volume des données générées par les institutions financières ont profondément transformé les méthodes de gestion et de pilotage stratégique au sein du secteur bancaire. Dans ce contexte d'hyper-digitalisation, la donnée s'est imposée comme le socle fondamental sur lequel reposent l'évaluation des risques, la personnalisation des services clients, et la conformité aux exigences réglementaires de plus en plus strictes (telles que la loi 09-08 de la CNDP au Maroc, le RGPD européen, ou encore la norme BCBS 239). Toutefois, malgré la mise en place d'infrastructures Big Data complexes telles que les Data Lakes pour centraliser ces informations, la seule capacité de stockage et d'ingestion ne suffit plus. Les données brutes restent souvent inexploitable ou dangereuses pour la prise de décision si elles s'avèrent incomplètes, incohérentes ou obsolètes.

Parallèlement, les progrès réalisés dans le domaine de l'ingénierie des données et de l'automatisation logicielle ouvrent de nouvelles perspectives pour assainir ces écosystèmes. Des outils modernes de validation des données, couplés à la puissance de calcul distribué (comme Apache Spark) et à des mécanismes d'orchestration avancés, permettent désormais d'automatiser l'audit continu des flux d'information. Ces technologies permettent non seulement de détecter les anomalies dès la source, d'enrichir les métadonnées de qualité, mais aussi de fournir des indicateurs de performance fiables (KPIs) avant que les données erronées n'impactent les modèles analytiques ou les reportings réglementaires en aval.

C'est dans cette optique que s'inscrit ce projet de fin d'études réalisé au sein du Data Office de CIH BANK. Son objectif consiste à concevoir, développer et industrialiser un pipeline automatisé de contrôle de la qualité des données (Data Quality) intégré directement à l'infrastructure Hadoop de la banque. En ciblant prioritairement le domaine stratégique des Tiers (incluant les informations d'identification KYC et les catégories socio-professionnelles CSP), le but est d'automatiser la validation, la surveillance et le monitoring de la qualité des données à travers des règles métiers strictes.

La solution développée repose sur une architecture Big Data complète comprenant l'ingestion massive des données depuis les systèmes opérants via Apache Sqoop, l'analyse et la validation des contenus à l'aide du framework Soda Core intégré à Apache Spark, ainsi que le stockage structuré des résultats dans des tables Hive. Au-delà du contrôle fonctionnel, ce projet met également l'accent sur la virtualisation des données via Dremio et la restitution interactive des métriques de qualité à travers un tableau de bord développé sous Tableau Desktop. Une chaîne d'orchestration complète basée sur Apache Airflow a été mise en œuvre afin d'automatiser ce cycle d'audit quotidiennement, garantissant ainsi une détection proactive des anomalies, une réduction des interventions manuelles chronophages, et une fiabilité accrue lors de l'exploitation des données par les métiers.

Le présent mémoire est organisé en quatre chapitres : Le premier chapitre présente le contexte général du projet, l'organisme d'accueil CIH BANK, l'étude de l'infrastructure Big Data existante ainsi que la méthodologie de travail et le planning prévisionnel adoptés. Le deuxième chapitre est consacré à l'analyse détaillée des besoins fonctionnels et non fonctionnels, suivie de la modélisation formelle des cas d'utilisation du système et de

leurs acteurs. Le troisième chapitre détaille la conception de la solution, comprenant la modélisation de l'activité, la conception des modèles conceptuels, logiques et relationnels de données, le catalogue exhaustif des règles de qualité par dimension, ainsi que l'architecture logicielle (diagramme de composants) et matérielle (diagramme de déploiement UML 2.5). Enfin, le quatrième chapitre présente la réalisation et l'implémentation technique de la solution (scripts PySpark et Soda Core, tables Hive, DAG Airflow et dashboards Tableau), avant de conclure sur le bilan du projet et ses perspectives d'évolution.

Chapitre 1

Présentation du cadre de projet

Introduction

Dans ce chapitre, nous présenterons une description générale de l'organisme d'accueil, CIH BANK, d'un point de vue organisationnel. Nous aborderons ensuite l'étude de l'existant, le choix du modèle de développement ainsi que le planning prévisionnel du projet, afin de mieux contextualiser notre travail de fin d'études.

1.1 Présentation de l'organisme d'accueil

Le Crédit Immobilier et Hôtelier (CIH), plus communément appelé CIH BANK, est une banque marocaine dont le siège social se trouve à Casablanca. Historiquement connu pour son activité dans le secteur de l'immobilier, le CIH propose désormais des services bancaires variés à ses clients, particuliers et professionnels. [10]

CIH BANK a été fondée en 1920 sous le nom de Caisse de Prêts Immobiliers du Maroc (CPIM). En 1967, le groupe a étendu son activité au secteur hôtelier et a changé de nom pour devenir le Crédit Immobilier et Hôtelier. [10]

Création	1920
Fondateur	L'État marocain
Introduction en bourse	1967
Forme juridique	Société anonyme
Slogan	La banque de demain dès aujourd'hui
Siège social	187, Avenue Hassan II, Casablanca, Maroc
Président Directeur Général (PDG)	Lotfi SEKKAT
Société mère	Caisse de Dépôt et de Gestion
Effectif	2 181
Site web	www.cihbank.ma

TABLE 2 – Informations générales sur le groupe CIH BANK [10]

1.1.1 Missions de CIH BANK

Spécialisé dans le financement immobilier et hôtelier, le CIH est un partenaire historique des pouvoirs publics en matière de financement du logement depuis 1920. La banque finance les promoteurs immobiliers au Maroc par deux canaux complémentaires. En amont, elle assure le financement des promoteurs publics et privés pour l'acquisition, la viabilisation des terrains et la construction des programmes immobiliers. En aval, elle octroie des crédits au logement pour les particuliers acquéreurs, tout en proposant divers services et produits bancaires adaptés à sa clientèle.

1.1.2 Secteurs d'activité

Le groupe CIH BANK et ses filiales offrent une vaste gamme de produits et de services couvrant les besoins de ses clients particuliers et professionnels. Ces services incluent les opérations bancaires classiques, les prêts aux particuliers (tels que les prêts immobiliers et crédits à la consommation), ainsi que le financement d'entreprises (via des prêts d'investissement, d'exploitation et du crédit-bail). Le groupe intervient également dans le domaine de l'assurance et dispose d'une salle des marchés active.

1.1.3 Filiales

Afin de diversifier ses activités, le groupe CIH BANK s’appuie sur plusieurs filiales spécialisées. On y retrouve SOFAC pour les solutions de crédit, SOFASSUR et CIH Courtage pour les services d’assurance, ainsi que Maroc Leasing, qui propose des solutions de financement pour les véhicules, le BTP, l’immobilier et la santé [6]. Le groupe est également présent dans la finance participative avec Ummia Bank, et dans la gestion d’Organismes de Placement Collectif Immobilier (OPCI) via AJAR INVEST, dont CIH BANK détient 40 % des parts [1].

1.1.4 Actionnariat

La répartition du capital du groupe CIH BANK est présentée dans le tableau 3 :

TABLE 3 – Répartition du capital de CIH BANK [4]

Actionnaire	Part
Massira Capital Management	55,66 %
Flottant en bourse	16,37 %
Atlanta Sanad	11,61 %
Caisse de Dépôt et de Gestion (CDG)	6,69 %
Régime Collectif d’Allocation de Retraite (RCAR)	5,21 %
Personnel de la banque	4,36 %
Groupe Holmarcom	0,12 %

1.1.5 Chiffres clés

La figure 1 présente les chiffres clés de CIH BANK.

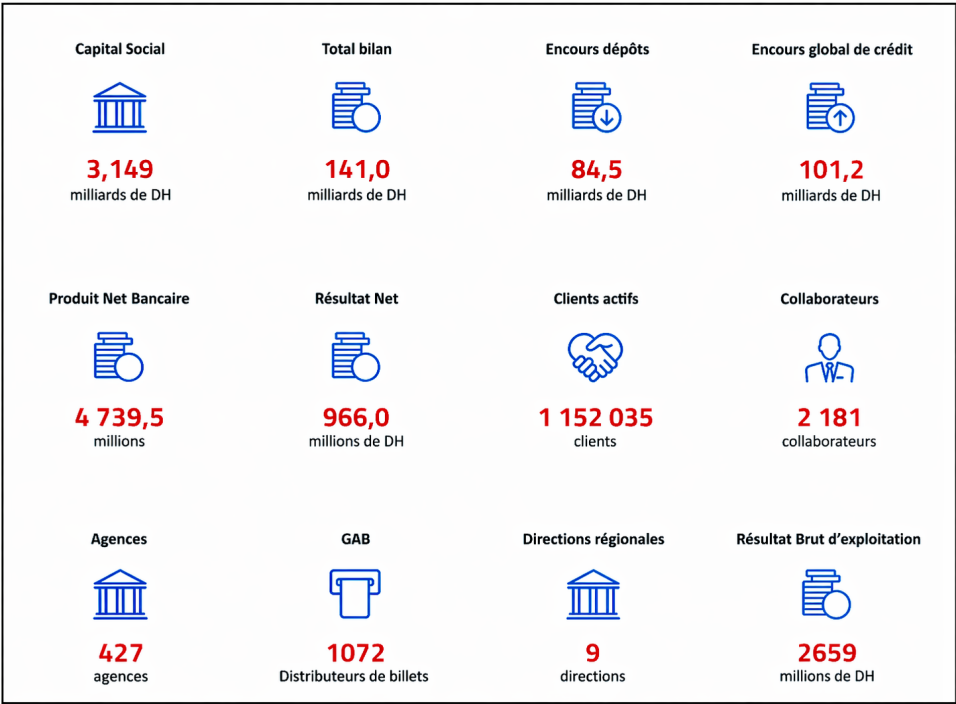


FIGURE 1 – Chiffres clés de CIH BANK (31/12/2024)

Les chiffres clés de CIH BANK reflètent la situation financière et opérationnelle de l'institution. Avec un capital social de 3,149 milliards de dirhams et un total bilan de 141 milliards de dirhams, la banque dispose d'une base financière significative. L'encours des dépôts s'élève à 84,5 milliards de dirhams, tandis que l'encours global de crédit atteint 101,2 milliards de dirhams. Le produit net bancaire est de 4 739,5 millions de dirhams, avec un résultat net de 966 millions de dirhams et un résultat brut d'exploitation de 2 659 millions de dirhams. La banque compte plus de 1,15 million de clients actifs, 2 181 collaborateurs, 427 agences, 1 072 guichets automatiques et 9 directions régionales. Ces indicateurs fournissent une vue d'ensemble de l'activité, de la structure et des résultats financiers de l'institution.

1.1.6 Organigramme

1.1.6.1 Structure hiérarchique de CIH BANK

La structure hiérarchique de CIH BANK s'articule autour d'une direction générale pilotée par le Président Directeur Général, appuyée par des fonctions de support stratégique et de contrôle (Audit, Conformité, Risques, Capital Humain, etc.) ainsi que diverses fonctions rattachées. L'activité commerciale et opérationnelle de la banque est quant à elle répartie en plusieurs pôles métiers majeurs : la Banque des Particuliers et Professionnels, la Banque de l'Entreprise et de l'Immobilier, la Banque de l'Investissement, le Financement & Recouvrement, et les Services à la Clientèle & Réseaux.

Parmi ces différentes directions, notre attention se porte plus particulièrement sur l'entité technologique qui encadre notre organisme d'accueil :

Fonctions technologiques et data Au niveau des fonctions technologiques et de gestion de la donnée, le pôle Services Technologiques et Opérations englobe la Sécurité des Systèmes d'Information, la Qualité, ainsi que le Pôle Systèmes d'Information. C'est au sein de la direction Data Management & Intelligence Artificielle de ce même pôle que se trouve le Data Office, l'entité d'accueil de notre projet de fin d'études.

L'organigramme global de CIH BANK est illustré dans la figure 2 ci-dessous.

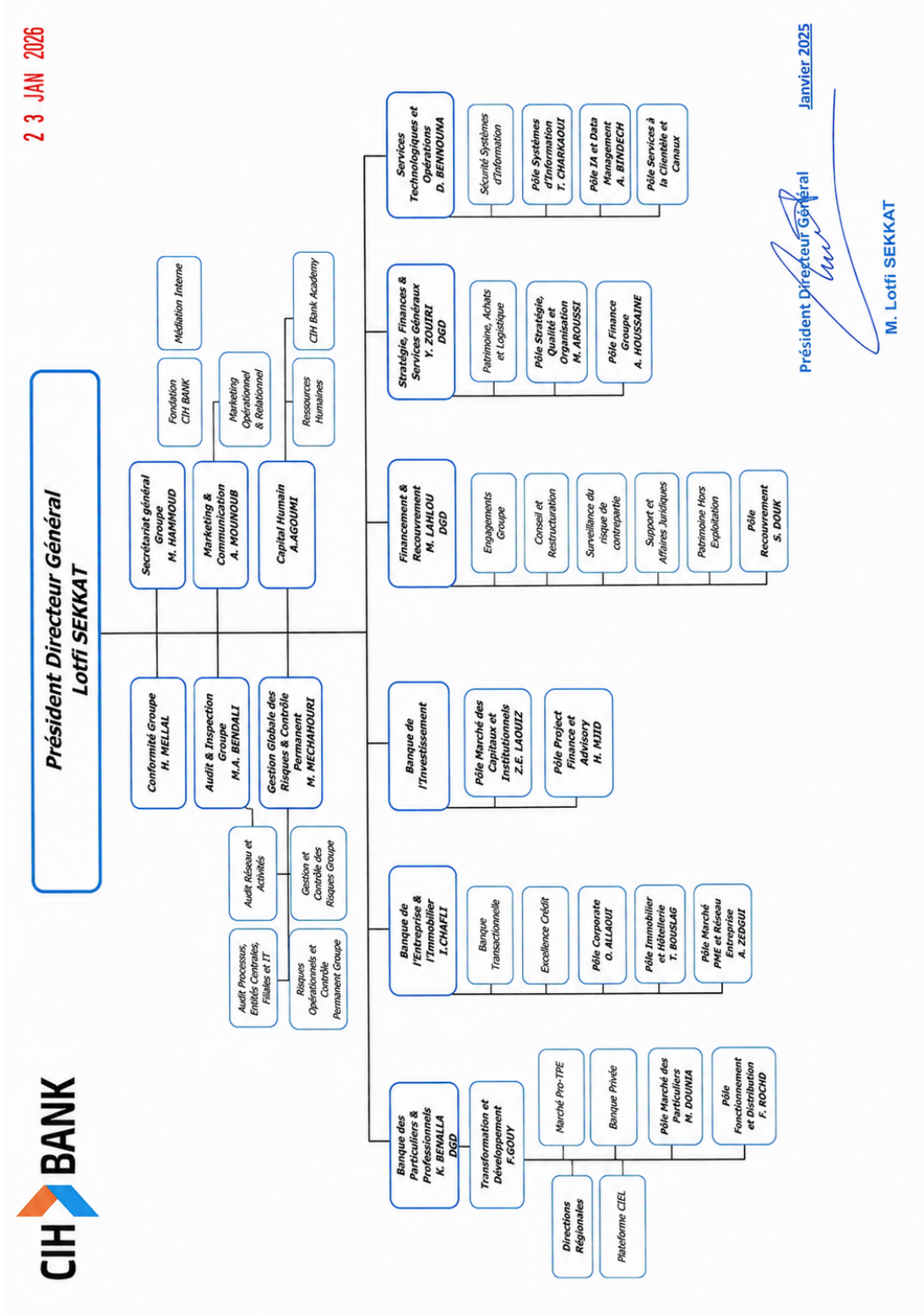


FIGURE 2 – Organigramme de CIH BANK

1.1.7 Data Office

Dans le cadre de notre stage, nous avons intégré le Data Office de CIH BANK, une entité dédiée à la gestion, au traitement et à la valorisation des données de la banque.

Le Data Office est aujourd'hui rattaché au pôle Services Technologiques et Opérations, plus précisément au sein de la direction Data Management et Intelligence Artificielle. Cette organisation reflète l'importance stratégique accordée à la donnée dans le pilotage des

activités de la banque.

Cette entité regroupe plusieurs profils complémentaires intervenant sur l'ensemble du cycle de vie de la donnée. Les *Data Engineers* sont en charge de l'ingestion, du traitement et de la mise à disposition des données. Ils collaborent étroitement avec les *Data Analysts*, responsables de l'analyse et de la production d'indicateurs décisionnels, ainsi qu'avec les *Data Scientists*, spécialisés dans la modélisation avancée et l'analyse prédictive. Enfin, l'équipe *Data Management* assure la gouvernance transversale, la qualité et la structuration des données au sein de la banque.

Les données sont collectées à partir des différents systèmes opérationnels de la banque, puis intégrées et centralisées au sein d'un Data Lake. Elles sont ensuite transformées et mises à disposition pour répondre aux besoins analytiques, réglementaires et métiers.

Dans ce cadre, le Data Office joue un rôle central dans la mise en place des pratiques de gestion de la qualité des données.

1.2 Étude de l'existant

1.2.1 Description de l'existant

Contexte et enjeux de la qualité des données bancaires

Dans l'écosystème financier contemporain, la transition vers une économie numérique a redéfini la nature des institutions bancaires. La donnée n'est plus un simple sous-produit de l'activité transactionnelle, mais s'impose comme une infrastructure essentielle sur laquelle reposent l'évaluation des risques, les décisions stratégiques, et une conformité réglementaire extrêmement stricte. Au Maroc, cette conformité s'articule prioritairement autour de la protection des données à caractère personnel, régie par la loi 09-08 de la CNDP (Commission Nationale de contrôle de la protection des Données à caractère Personnel). S'y ajoute le respect du RGPD européen pour la clientèle résidant en Europe, ainsi que d'autres normes internationales liées aux risques (ex : BCBS 239). Sur le plan économique, le coût de la non-qualité est très élevé. La « règle de dix » (*Rule of Ten*) stipule qu'il coûte dix fois plus cher de corriger une donnée en aval que de la valider directement à la source. Une gouvernance proactive apporte une valeur mesurable : optimisation des fonds propres, efficacité opérationnelle et sécurisation des décisions basées sur l'intelligence artificielle.

Le domaine Tiers : Cartographie et données

Historiquement, la vérification de la qualité des données au sein de CIH BANK était un processus réactif déclenché sur demande, ce qui a motivé la création de l'entité Data Management (DM). Notre projet s'inscrit dans cette volonté d'amélioration continue et cible plus particulièrement le **Référentiel Tiers**. Ce référentiel regroupe l'ensemble des informations relatives aux personnes physiques et morales en relation avec la banque.

Dans le cadre de l'ingestion des données vers le Data Lake, l'équipe travaille sur plus d'une dizaine de tables du domaine Tiers. À titre d'illustration, les tables principales extraites du système opérant Oracle NOVA (CCM) qui structurent notre périmètre de base sont : **identity** (pièces d'identité), **legalentity** (personnes morales), **person** (statut client), **professionaldescription** (caractéristiques professionnelles), **jobs** (catégories socio-professionnelles CSP), ainsi que la table centrale **np_description** (Natural Person Description) regroupant les données d'état civil et les informations KYC pour les personnes physiques.

Bien que le pipeline automatisé de qualité de données déployé couvre l'ensemble du domaine Tiers (soit une quinzaine de tables issues de NOVA), **pour des soucis de clarté et d'illustration, les implémentations techniques et les règles de Data Quality présentées dans la suite de ce rapport se focaliseront particulièrement sur les tables np_description et jobs**. Ces tables sont éminemment stratégiques : np_description concentre les données KYC indispensables à l'identification réglementaire du client (nom, prénom, date de naissance, genre, nationalité, etc.), tandis que jobs contient des informations essentielles pour la catégorisation socio-professionnelle (CSP).

Architecture et pipeline de données

L'ensemble des données ingérées au sein de CIH BANK, incluant celles du domaine Tiers, transite par une plateforme robuste centralisée s'appuyant sur une infrastructure Big Data de pointe. Tout l'écosystème analytique repose sur ce système unifié pour le traitement et la valorisation de la donnée.

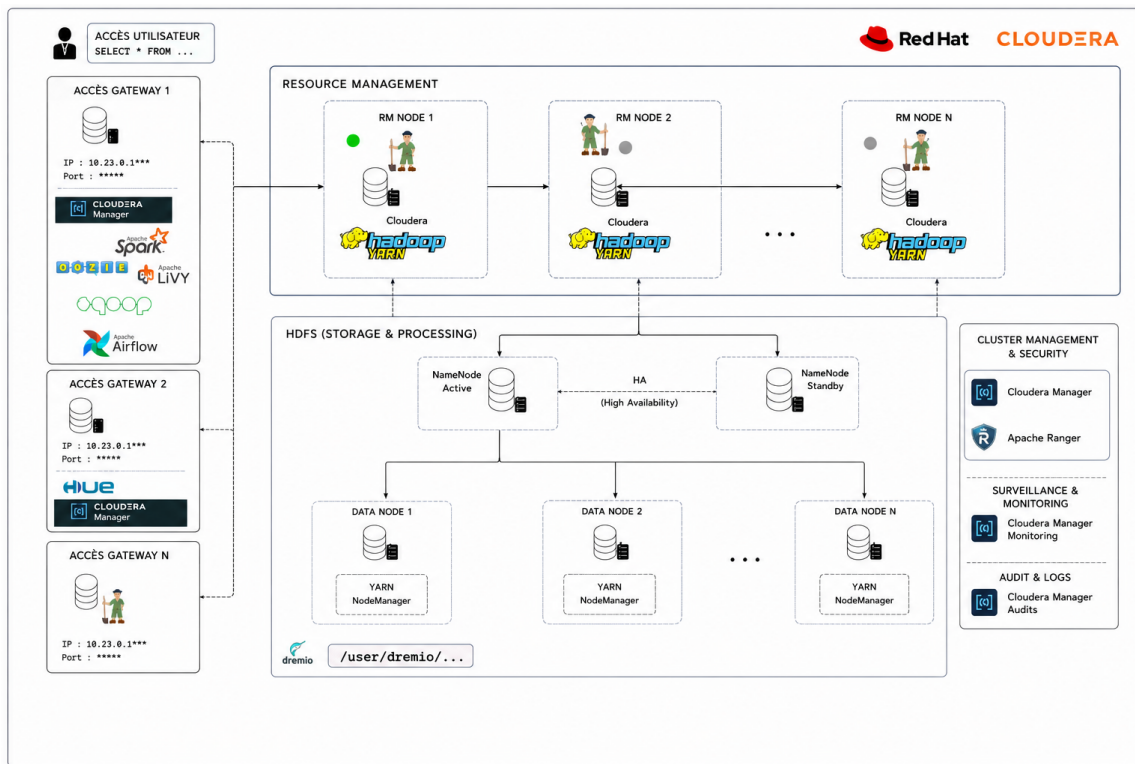


FIGURE 3 – Architecture matérielle Big Data

L'architecture matérielle et logicielle de cette plateforme de production s'articule autour d'un **cluster Cloudera et Dremio** composé de 8 nœuds (3 Masters, 3 Workers, 2 Gateways). Le flux global des données, de la source à l'exploitation, suit un cycle de vie standardisé en plusieurs étapes :

- 1. Ingestion** : Les données brutes issues des systèmes opérants (Oracle NOVA, PowerCard, etc.) sont ingérées massivement via des scripts **Apache Sqoop** organisés par domaine métier. Les tables sont stockées au format **ORC** dans la zone RAW de Hive via HCatalog. L'authentification aux sources s'effectue par **Kerberos** (kinit avec keytab).
- 2. Transformation** : Des pipelines PySpark packagés sous forme de fichiers **Wheel** (.whl), construits via le gestionnaire **PDM**, effectuent le nettoyage, le renommage des

colonnes et l'enrichissement des données brutes pour les charger dans la zone de STAGING. Chaque écriture est tracée par une colonne d'audit `batch_time`.

3. Virtualisation et Métadonnées : Le moteur de données Dremio se connecte aux catalogues (Hive Metastore) pour virtualiser les tables. Il offre une couche sémantique puissante permettant de croiser et d'interroger les données sans les dupliquer.

4. Exposition et Datamarts : Les données transformées sont consolidées et exposées sous forme de Datamarts (ou vues métiers spécifiques). Ces ensembles de données prêts à l'emploi alimentent les outils de restitution (Tableau) pour répondre aux besoins analytiques des métiers.

1.2.2 Critique de l'existant

Bien que cette infrastructure de production Cloudera soit robuste, l'absence d'un contrôle automatisé de la qualité des données en amont pose plusieurs problèmes majeurs transverses à l'ensemble des tables du domaine Tiers. Pour illustrer l'impact de ces problèmes, nous prenons l'exemple des tables `np_description` et `jobs`, qui contiennent des données KYC et CSP particulièrement sensibles concernant les personnes physiques :

Absence de validation à la source : Les données transitent vers Dremio sans aucune validation intermédiaire. Des incohérences flagrantes persistent, comme des discordances entre le genre et la civilité du client, ou des dates de naissance remplies par défaut.

Risques de non-conformité réglementaire : L'absence ou l'invalidité de certains champs critiques, tels que l'identifiant FATCA pour les clients concernés, expose la banque à des risques réglementaires et à d'éventuelles sanctions.

Démarche réactive et manuelle : Lorsqu'une anomalie est repérée par les métiers en bout de chaîne, la réconciliation implique asynchronement plusieurs équipes pour des corrections curatives chronophages, créant ainsi des processus inefficaces.

Manque de traçabilité et d'historisation : Aucune métrique n'est historisée dans le temps. Les Data Stewards ne disposent d'aucun outil pour prioriser ou surveiller la santé des données KYC au quotidien.

1.2.3 Solution proposée

Pour pallier ces limites, nous proposons de concevoir et d'implémenter un **pipeline automatisé de contrôle de la qualité des données** couvrant l'ensemble du périmètre Tiers, intégré directement dans la zone RAW de l'infrastructure Hadoop. La solution technique s'articule autour de trois piliers majeurs :

Automatisation native via Soda Core : L'intégration de la bibliothèque Soda Core au sein des workflows Airflow existants permet de valider systématiquement les données KYC (exhaustivité, unicité, validité de format, cohérence croisée) dès leur ingestion.

Approche proactive et détection précoce : En signalant les anomalies dès la zone RAW, nous empêchons l'exploitation de données erronées par les couches analytiques aval, appliquant ainsi le principe de la *Rule of Ten*.

Monitoring centralisé et restitution décisionnelle : Les résultats des contrôles et les anomalies consolidées sont persistés dans des tables Hive dédiées. Un tableau de bord interactif est ensuite construit pour offrir aux Data Stewards une vue claire de la santé des tables du référentiel, tout en garantissant la stricte anonymisation des données personnelles.

Cette solution permet d'améliorer considérablement la gouvernance des données du domaine Tiers tout en optimisant le temps de traitement des anomalies.

1.3 Choix du modèle de développement

Pour mener à bien ce projet, nous avons adopté une démarche **Agile inspirée du framework Scrum, adaptée aux spécificités de l'ingénierie Big Data**. La complexité de l'écosystème distribué et l'étendue du domaine Tiers ont motivé un découpage itératif par livrables fonctionnels (*Minimum Viable Products* et *Proof of Concepts*), permettant d'éprouver et de valider les règles de qualité de manière progressive, table par table.

Sur le plan opérationnel, cette démarche s'est articulée autour d'un rythme adapté : des synchronisations hebdomadaires (*Weekly standups*) avec le *Delivery Manager* pour piloter l'avancement des *backlogs* individuels et lever les verrous techniques, associées à des points de validation réguliers avec le *Data Steward* (agissant en tant que *Product Owner* métier). Cette organisation a assuré une forte réactivité face aux contraintes de l'infrastructure Cloudera tout en maintenant un alignement continu avec les priorités métiers de la banque.

1.4 Planning prévisionnel

Afin de mener à bien ce projet au sein du Data Office de CIH BANK, nous avons défini un planning de travail couvrant une période de six mois, du **9 février au 9 août 2026**.

Contrairement à un projet classique, l'avancement n'a pas été strictement linéaire. Les deux premiers mois ont été presque exclusivement dédiés à la compréhension fonctionnelle, technique, et à la documentation détaillée de l'architecture existante. Par la suite, les phases de conception, de réalisation des POCs, et de tests se sont superposées de manière itérative, accompagnées de réunions continues avec les métiers.

La figure 4 ci-dessous illustre ce déroulement sous forme d'un diagramme de Gantt, mettant en évidence la phase initiale de documentation, le cycle itératif de développement, et l'implication constante des acteurs métiers.

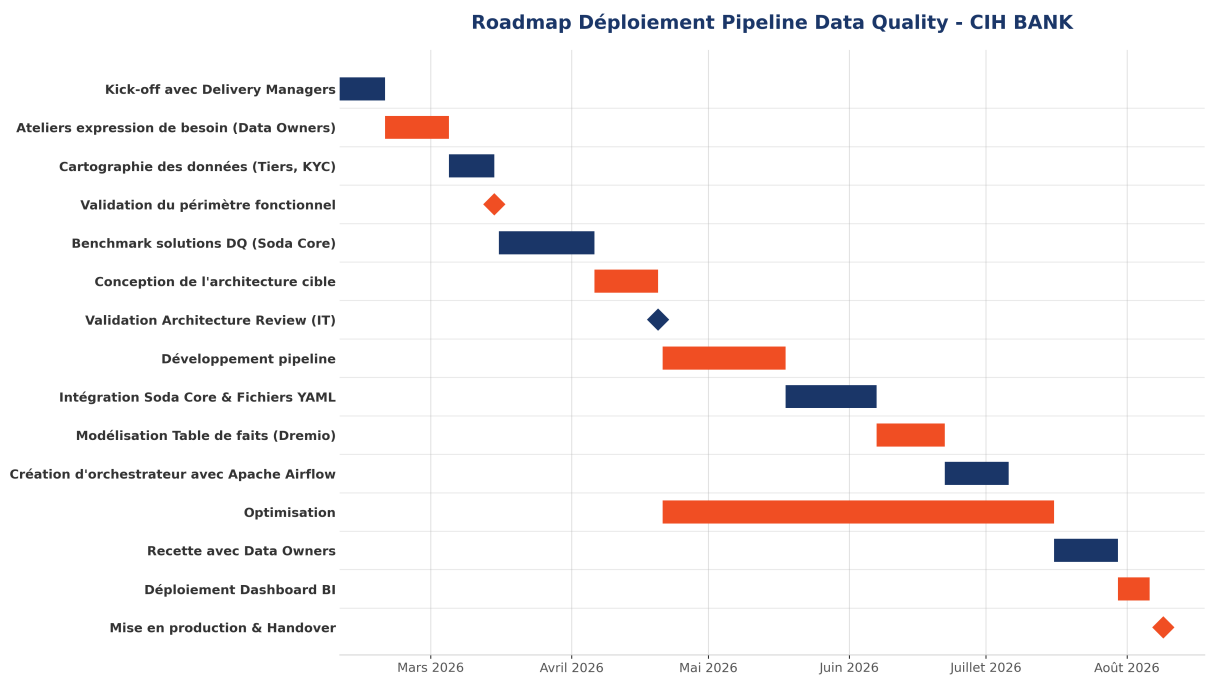


FIGURE 4 – Planning prévisionnel du projet

Conclusion

Pour conclure, CIH BANK est une institution bancaire marocaine majeure, fortement structurée et engagée dans la valorisation de la donnée à travers son Data Office. L'étude de l'existant a permis de mettre en évidence les processus actuels de gestion des données et leurs limites, ce qui justifie la nécessité d'un pipeline automatisé de contrôle qualité. Ces éléments constituent la base de notre projet et nous permettent d'entamer sereinement la phase de spécification des besoins dans le chapitre suivant.

Chapitre 2

Analyse des Besoins et Spécification

Introduction

Après avoir décrit le cadre général du projet et analysé le fonctionnement de la solution existante au sein de CIH BANK, ce chapitre se focalise sur la définition claire et rigoureuse des besoins. Nous y détaillons d'une part les besoins fonctionnels, c'est-à-dire les fonctionnalités attendues du pipeline de Data Quality, et d'autre part les besoins non fonctionnels, liés aux exigences techniques, de performance et de sécurité. Enfin, nous présentons la modélisation fonctionnelle à travers la spécification des cas d'utilisation et de leurs acteurs.

2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions que le système doit être capable de réaliser pour répondre aux problématiques de qualité des données identifiées sur le domaine Tiers. Ces problématiques, transverses à l'ensemble des tables du référentiel, seront principalement illustrées par les tables `np_description` (et ses 40 colonnes telles que `name`, `firstname`, `civility`, `fatcaidentificationnumber`, etc.) et `jobs`. Les besoins se répartissent en quatre fonctionnalités majeures :

2.1.1 Automatisation et planification des contrôles

Déclenchement automatique : Le système doit lancer l'évaluation de la qualité des données de manière planifiée, immédiatement après l'exécution complète du batch quotidien d'ingestion des tables sources Oracle NOVA (qui concernent environ 15 tables préfixées par `CCM_*` pour le périmètre Tiers) via un nouveau DAG dédié nommé `dq_tiers_daily00` s'exécutant à 00h00, séparé du DAG d'ingestion `dailyow22` de 22h00.

Orchestration intégrée : L'exécution du scan de qualité doit être orchestrée au sein d'Apache Airflow, permettant de suivre son statut d'exécution (Succès/Échec) et de consulter les journaux de bord en temps réel.

2.1.2 Configuration déclarative des règles de qualité

Moteur de règles dynamique : Le système doit permettre de définir les contrôles de qualité (Missing, Invalid, Freshness, Consistency, Uniqueness) dans des fichiers de règles déclaratifs séparés du code source applicatif.

Graduation des seuils et des alertes : Pour chaque contrôle, le système doit pouvoir évaluer des seuils paramétrables (par exemple un taux de valeurs manquantes inférieur à 5%) et générer un statut de warning ou d'error selon la gravité de l'écart constaté.

Gestion des métadonnées : Chaque règle doit être documentée avec des attributs métier précis (nom de la règle, dimension DQ associée, domaine KYC ou CSP, criticité fonctionnelle).

2.1.3 Historisation et traçabilité des indicateurs

Persistance journalière : À chaque exécution, le système doit sauvegarder les métriques calculées et les résultats de chaque contrôle de qualité dans des tables d'historique de scores (base Hive dédiée `dq_results`), indexée par date d'exécution.

Suivi global des anomalies : Le système doit capturer de manière consolidée les volumes et les pourcentages d'enregistrements en échec pour chaque contrôle de qualité. Cette approche permet de quantifier et d'isoler les axes de qualité défaillants sans avoir à stocker les données personnelles des clients concernés dans le système de Data Quality.

2.1.4 Restitution visuelle et aide à la décision

Exposition des données : Les résultats d'historisation de la qualité doivent être exposés de manière transparente sous forme de vues SQL pour les outils décisionnels.

Suivi de l'évolution temporelle : Le tableau de bord doit permettre de suivre la tendance d'évolution des scores DQ dans le temps (sur les 30 derniers jours).

Priorisation des axes de remédiation : Le tableau de bord doit mettre en évidence les contrôles les plus critiques en échec et les dimensions de qualité les plus impactées, orientant ainsi les équipes métier vers les chantiers prioritaires de nettoyage dans le système source NOVA.

2.2 Besoins non fonctionnels

Les besoins non fonctionnels expriment les exigences qualitatives, opérationnelles et techniques que la solution doit respecter pour être viable et performante dans l'environnement de production de CIH BANK :

2.2.1 Performance

Le Référentiel Tiers de la banque regroupe **plusieurs millions** de dossiers clients (personnes physiques). Les règles de cohérence inter-colonnes ou de doublons nécessitent des jointures lourdes. Le moteur DQ doit être capable d'interroger et d'évaluer ce volume massif de données en un temps minimal (moins de 15 minutes) sans dégrader les ressources du Data Lake. Cela justifie l'utilisation d'un moteur de calcul distribué (Apache Spark) pour le traitement.

2.2.2 Disponibilité

L'exécution des contrôles de qualité ne doit pas bloquer la disponibilité des données pour les outils de reporting en cas d'anomalie. Si un seuil de qualité est franchi ou si le scan échoue, le pipeline d'exposition (Dremio/Tableau) doit continuer à fonctionner en mode dégradé en signalant l'alerte. De plus, la tâche DQ doit s'exécuter même si certaines tables d'ingestion n'ont été chargées que partiellement.

2.2.3 Maintenabilité

L'ajout de nouvelles règles ou l'intégration de nouvelles colonnes du Référentiel Tiers (par exemple pour de nouveaux critères réglementaires ou pour les personnes morales) doit se faire sans modification du code Python principal de la solution. Les fichiers de configuration YAML et la structure des tables Hive de résultats doivent être conçus de manière générique pour supporter l'extensibilité.

2.2.4 Sécurité

Les détails d'anomalies de qualité des données contiennent des informations clients sensibles (`name`, `firstname`, radicaux). Seuls les collaborateurs habilités du Data Office (Data Stewards, Data Project) et les Data Owners des départements concernés doivent être autorisés à accéder aux détails des anomalies. La solution s'appuie sur les mécanismes de sécurité existants de la plateforme :

Authentification Kerberos : L'ensemble des composants du cluster Cloudera (HDFS, Hive, Spark) sont sécurisés par Kerberos, garantissant l'identité de chaque service et utilisateur.

Autorisation Apache Ranger : Les politiques d'accès aux tables Hive et aux espaces HDFS sont gérées de manière centralisée via Ranger, permettant un contrôle fin par rôle.

Annuaire LDAP / Active Directory : L'authentification des utilisateurs Dremio et Tableau est déléguée à l'annuaire d'entreprise, assurant une gestion unifiée des identités.

2.2.5 Conformité Réglementaire et Normative

Le système de gestion de la qualité des données doit s'inscrire dans un cadre strict de conformité légale et normative. Concernant la confidentialité et le traitement des données personnelles (noms, dates de naissance, identifiants), la solution doit impérativement respecter les exigences de la **loi 09-08** promulguée par la CNDP (Commission Nationale de contrôle de la Protection des Données à caractère personnel) au Maroc. De plus, la banque gérant potentiellement des clients résidant en Europe, la solution s'aligne sur les directives rigoureuses du **RGPD** (Règlement Général sur la Protection des Données). Enfin, la démarche globale de fiabilisation et de sécurisation de l'information s'appuie sur les standards internationaux, notamment la norme **ISO 8000** (standard mondial pour la qualité des données) et la norme **ISO/IEC 27001** (management de la sécurité de l'information).

2.3 Modélisation des cas d'utilisation

2.3.1 Acteurs du système

L'intégration de la solution de Data Quality au sein de CIH BANK repose sur une étroite collaboration entre les experts métier, l'équipe technique et les systèmes d'information. Le tableau 4 présente les cinq acteurs clés du système.

TABLE 4 – Acteurs du système de Data Quality

Acteur	Type	Rôle dans le système DQ
Équipe Data (DE / Data Steward)	Interne	Organise les ateliers d'alignement, formalise les règles dans les fichiers YAML Soda Core et implémente les flux techniques.
Métiers (Data Owners)	Humain	Experts fonctionnels qui définissent et valident les règles de qualité, consultent le dashboard et remédient aux anomalies.
Équipe SI	Appui	Valide la faisabilité technique des règles vis-à-vis du modèle physique Oracle NOVA.
Airflow (Orchestrateur)	Système	Déclenche automatiquement le scan DQ chaque soir après l'ingestion (nouveau DAG à 00h00).
Oracle NOVA (Système source)	Externe	Récepteur final des corrections de données opérées par les gestionnaires suite aux alertes DQ.

2.3.2 Diagramme global des cas d'utilisation

Le diagramme ci-dessous synthétise graphiquement les cas d'utilisation du système de Data Quality et les interactions avec les différents acteurs impliqués dans ce cycle de gouvernance.

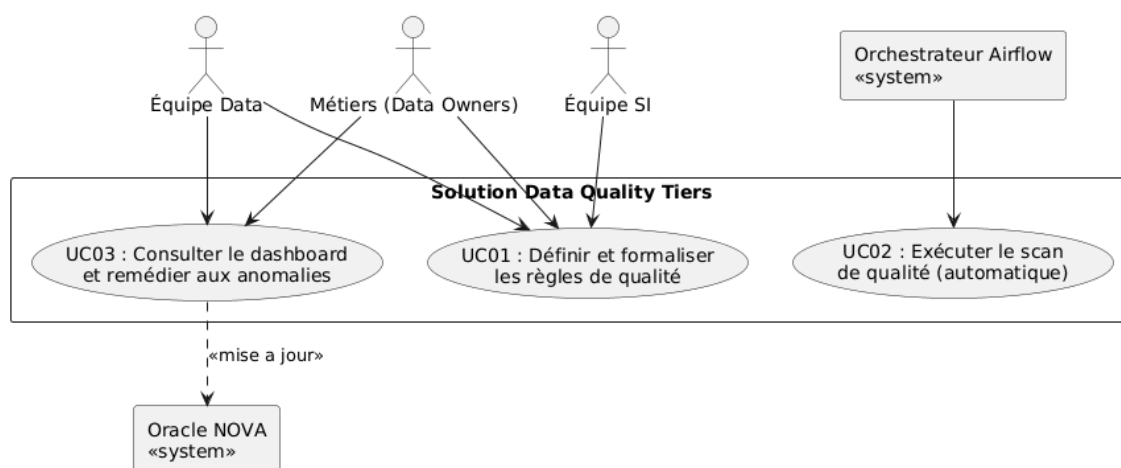


FIGURE 5 – Diagramme global des cas d'utilisation de la solution DQ Tiers

2.3.3 Description textuelle des cas d'utilisation principaux

Nous détaillons ci-dessous le déroulement des trois phases principales constituant le cycle de vie de la qualité des données de notre solution.

2.3.3.1 Cas d'utilisation 01 : Définir et formaliser les règles de qualité

TABLE 5 – Description du cas d'utilisation 01

Champ	Description
Titre	Définir et formaliser les règles de qualité
Acteurs	Équipe Data (Steward / DE), Métiers (Data Owners), Équipe SI
Préconditions	Cadrage fonctionnel du domaine Tiers défini
Scénario nominal	1. L'équipe Data organise des ateliers d'alignement avec les experts Métier et l'équipe SI. 2. Les parties s'accordent sur le catalogue des contrôles (complétude KYC, cohérence CSP) et les seuils de tolérance. 3. L'équipe Data consigne les spécifications dans un document de cadrage. 4. Le Data Engineer implémente les règles dans les fichiers YAML Soda Core.
Postconditions	Les règles de qualité sont formalisées et déployées dans le moteur Soda.

2.3.3.2 Cas d'utilisation 02 : Exécuter le scan de qualité (automatique)

TABLE 6 – Description du cas d'utilisation 02

Champ	Description
Titre	Exécuter le scan de qualité (automatique)
Acteur principal	Orchestrateur Airflow (Système)
Préconditions	Fin de l'ingestion quotidienne Sqoop des tables NOVA
Scénario nominal	1. Airflow lance automatiquement la tâche DQ de nuit. 2. Le moteur DQ Soda Core lit les fichiers YAML et exécute les requêtes sur Spark. 3. Les résultats agrégés et les détails des anomalies clients sont historisés dans la base Hive <code>dq_results</code>.
Postconditions	Les indicateurs et anomalies du jour sont prêts à être exposés.

2.3.3.3 Cas d'utilisation 03 : Consulter le dashboard et remédier aux anomalies

TABLE 7 – Description du cas d'utilisation 03

Champ	Description
Titre	Consulter le dashboard et remédier aux anomalies
Acteurs	Métiers (Parties prenantes), Data Stewards
Préconditions	Le scan DQ s'est exécuté de manière conforme
Scénario nominal	1. Le dashboard Tableau se rafraîchit automatiquement chaque matin. 2. Chaque partie prenante consulte le dashboard depuis son poste de travail. 3. Chaque acteur filtre et exporte la liste des anomalies de son périmètre. 4. Les acteurs procèdent à la correction des anomalies dans Oracle NOVA.
Postconditions	Les données corrigées dans NOVA sont ré-ingérées lors du prochain batch et le score DQ s'améliore automatiquement.

Conclusion

Ce chapitre a permis de formaliser les besoins fonctionnels et non fonctionnels, définissant de manière précise les objectifs techniques et opérationnels du pipeline de qualité des données. La modélisation des acteurs et des cas d'utilisation reflète fidèlement le cycle de gouvernance de CIH BANK, où chaque partie prenante collabore d'abord à la définition des contrôles, puis intervient individuellement sur son périmètre pour corriger les anomalies identifiées au quotidien. Fort de ces spécifications, le chapitre suivant présente la conception globale et détaillée de la solution.

Chapitre 3

Conception de la Solution

Introduction

Après avoir défini les besoins fonctionnels et non fonctionnels de notre système de contrôle de la qualité des données, ce chapitre présente la conception détaillée de la solution. Dans un premier temps, nous décrivons la modélisation dynamique à travers des diagrammes de séquences et d'activités. Dans un second temps, nous présentons la modélisation statique comprenant la modélisation de la table de faits, le modèle relationnel, le dictionnaire de données et le catalogue des règles de qualité (illustré sur deux tables exemplaires du périmètre). Enfin, nous détaillons l'architecture logicielle (diagramme de composants) et matérielle (diagramme de déploiement) de l'application dans son environnement CIH BANK.

3.1 Modélisation dynamique

La modélisation dynamique permet de représenter les interactions temporelles entre les différents composants du système et l'évolution des états des données au cours de l'exécution du processus de contrôle.

3.1.1 Diagramme de séquences

Le diagramme de séquences de la figure 6 détaille le flux d'exécution séquentiel d'un scan de qualité de données. Il illustre comment la tâche DQ intégrée dans Airflow coordonne le traitement entre la zone HDFS RAW, le moteur Soda Core et la base Hive de résultats, jusqu'à l'exposition finale vers Tableau.

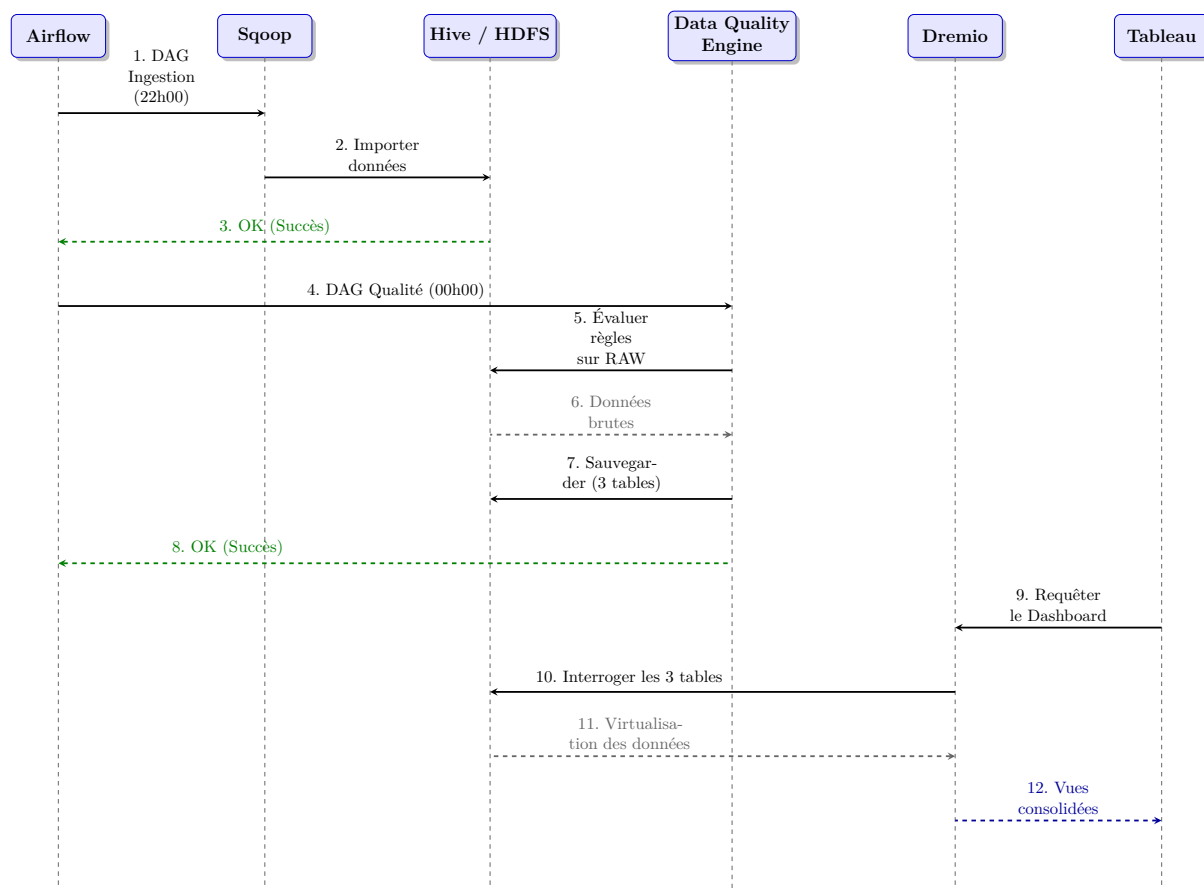


FIGURE 6 – Diagramme de séquences du pipeline de contrôle et restitution DQ

3.1.2 Diagramme d'activités

Le diagramme d'activités présenté dans la figure 7 décrit le processus de bout en bout de traitement de la qualité, depuis le chargement initial des données en zone RAW HDFS jusqu'à la remédiation effective dans l'application source Oracle NOVA.

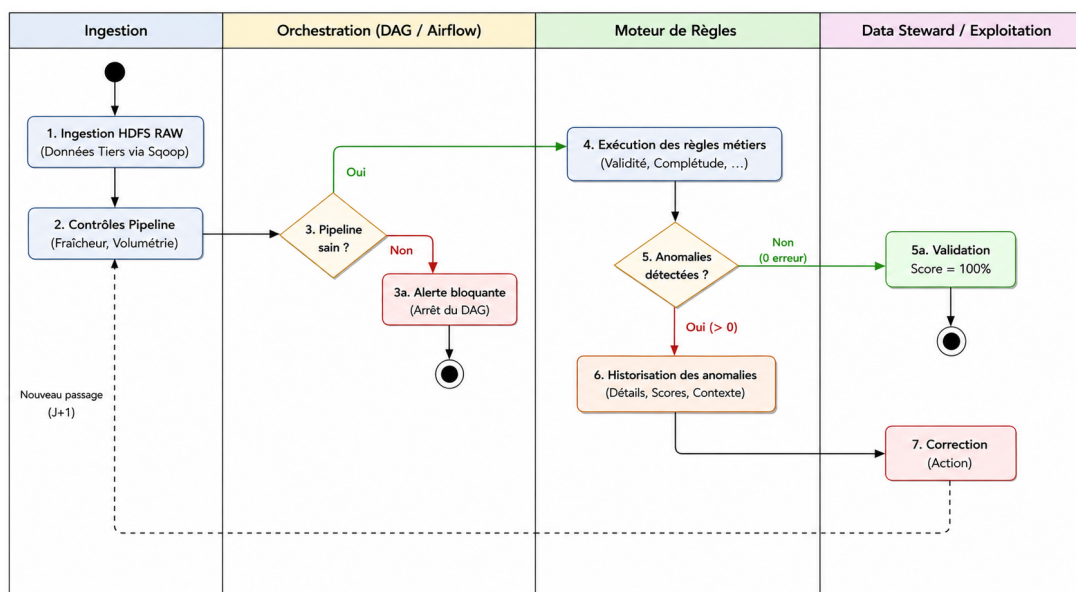


FIGURE 7 – Diagramme d'activités du workflow de remédiation DQ

3.2 Modélisation statique

La modélisation statique définit la structure fixe du système, ses composants de données, le schéma relationnel de stockage de l'historique de qualité, ainsi que le dictionnaire des métadonnées associées.

3.2.1 Modélisation de la table de faits

La figure 8 illustre la structure des données générée par notre moteur de qualité. Le moteur Soda Core produit un ensemble de trois tables physiques liées par l'identifiant d'exécution (`run_id` et `query_id`). Ces trois tables sont ensuite unifiées au sein de Dremio pour exposer une grande table de faits consolidée (21 colonnes) servant de source unique de vérité pour le reporting BI.

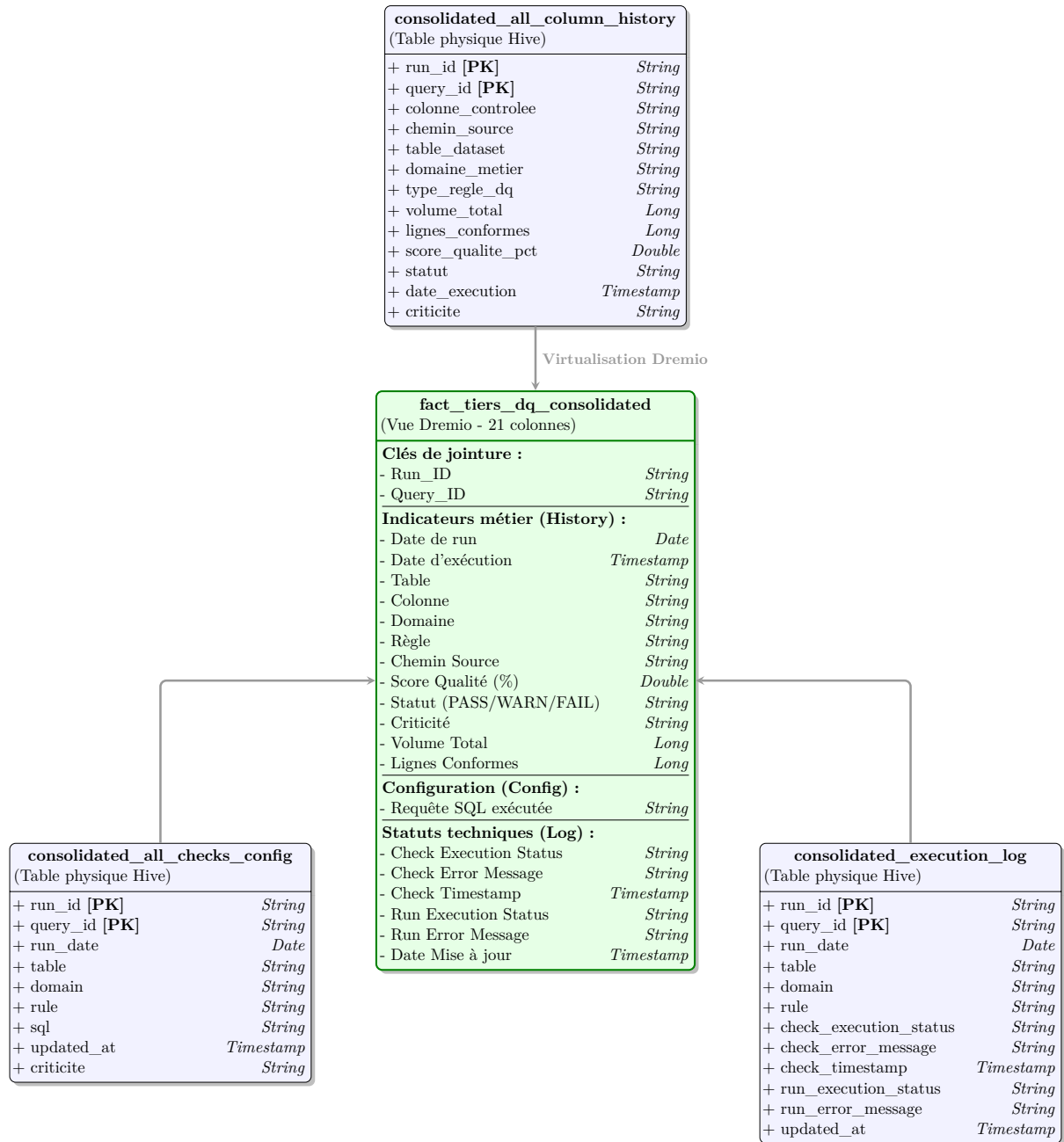


FIGURE 8 – Modélisation de la grande table de faits `fact_tiers_dq_consolidated` et ses 3 tables sources

3.2.2 Modèle relationnel

Pour persister les résultats d'évaluation de la qualité des données, nous concevons un schéma physique relationnel indépendant au sein de la base de données Hive, basé sur trois tables distinctes :

consolidated_all_column_history : Table principale d'historisation des indicateurs et scores de qualité par exécution, table et colonne. Elle inclut la criticité métier du contrôle.

consolidated_all_checks_config : Table de configuration stockant la requête SQL générée par le moteur pour chaque règle évaluée, ainsi que sa criticité.

consolidated_execution_log : Table de journalisation technique traçant le succès ou l'échec d'exécution du contrôle.

3.2.3 Dictionnaire de données

Les attributs clés de ces trois tables sont consolidés via Dremio pour fournir un ensemble de 21 colonnes à Tableau. Voici les colonnes principales exploitées :

TABLE 8 – Dictionnaire simplifié des tables de faits de Qualité des Données

Colonne	Table source	Description / Rôle
run_id	Toutes	Identifiant unique de l'exécution du processus DQ.
query_id	Toutes	Identifiant de la requête ou de la règle évaluée.
type_regle_dq	column_history	Dimension DQ (ex : Completude, Conformite, Coherence...).
criticite	column_history / checks_config	Niveau de criticité du contrôle (CRITIQUE, HAUTE, MOYENNE).
domaine_metier	column_history	Domaine métier visé (PTP-TIE-POR).
score_qualite_pct	column_history	Taux de conformité en pourcentage.
statut	column_history	État d'évaluation métier de la règle (PASS, WARN, FAIL).
sql	checks_config	Requête SQL exécutée par le moteur SparkSQL.
check_execution_status	execution_log	Statut d'exécution technique du job (SUCCESS/ERROR).

3.2.4 Catalogue des règles de qualité

Les contrôles de qualité s'appliquent sur l'ensemble des données du domaine Tiers (informations signalétiques, données socioprofessionnelles, etc.). Afin d'offrir une structuration claire et directement exploitable, le catalogue des règles de gestion est organisé par **dimension de Data Quality** : Complétude, Conformité, Cohérence, Unicité et Actualité / Pipeline.

L'établissement de ce catalogue découle d'une analyse exploratoire approfondie menée sur l'exhaustivité des dossiers physiques issus des tables du périmètre (**environ 15 tables CCM_***). Cette analyse macroscopique a révélé des anomalies typiques et récurrentes de saisie au sein du système opérant Oracle NOVA, ce qui a directement guidé le choix et le paramétrage de nos dimensions de contrôle :

Le phénomène des dates génériques : Une concentration anormale de dates de naissance fixées au premier janvier a été identifiée, traduisant une saisie par défaut dans le système lorsque la date exacte n'est pas connue. Cela justifie notre règle de validité temporelle avec un seuil de tolérance métier adapté.

Le codage des champs de référence : Des champs textuels (comme les villes) contiennent parfois des codes numériques issus d'anciens systèmes, nécessitant un contrôle strict par rapport au référentiel central pour assurer la cohérence des données.

La cohérence conditionnelle réglementaire (ex. FATCA) : Certains identifiants fiscaux internationaux doivent obligatoirement répondre à des règles de remplissage croisées

très strictes, dépendantes d'autres champs (comme le pays de nationalité ou de résidence fiscale).

3.2.4.1 Dimension Complétude

La dimension de complétude mesure le taux de renseignement des attributs obligatoires pour assurer l'exploitabilité opérationnelle des dossiers clients.

TABLE 9 – Dimension Complétude — Données Tiers (Référentiel Tiers)

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
name	0 valeur nulle tolérée	CRITIQUE	Audit dossiers - Blocage saisie agence
firstname	0 valeur nulle tolérée	CRITIQUE	Audit dossiers - Blocage saisie agence
birthdate	0 valeur nulle tolérée	CRITIQUE	Saisie obligatoire avec justificatif officiel
nationality_country	0 valeur nulle tolérée	HAUTE	Renseigner le code pays de nationalité
professional_sector	Taux de valeurs nulles < 5%	HAUTE	Renseigner le secteur d'activité de l'employeur
occupation	Taux de valeurs nulles < 5%	HAUTE	Préciser la profession exacte du client
socioprofessional_category	Taux de valeurs nulles < 5%	HAUTE	Renseigner la catégorie socioprofessionnelle (CSP)
professional_situation	Taux de valeurs nulles < 5%	HAUTE	Renseigner le statut de la situation professionnelle
employer	Taux de valeurs nulles < 10%	HAUTE	Renseigner le nom de l'entreprise employeuse

3.2.4.2 Dimension Conformité

La dimension de validité évalue le respect des formats de saisie, des plages de valeurs autorisées et la conformité par rapport aux référentiels métier (BAM, ISO).

TABLE 10 – Dimension Conformité — Données Tiers (Référentiel Tiers)

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
name, firstname	Longueur ≥ 2 , aucun chiffre ou caractère spécial	HAUTE	Nettoyage et normalisation dans NOVA
birthdate	Taux de dates génériques au 01/01 < 15%	HAUTE	Recherche de la date réelle sur pièce d'identité
civility	Valeurs restreintes à (M., MME, MLLE)	HAUTE	Restauration de la valeur selon pièces fournies
sex	Valeurs restreintes à (M, F)	HAUTE	Restauration de la valeur selon pièces fournies
birth_city	Doit exister dans le référentiel des villes BAM	MOYENNE	Mapping et correction via la table de transcodage

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
professional_sector	Code numérique valide dans le référentiel sectoriel	MOYENNE	Transcodage vers les codes officiels BAM
professional_situation	Valeurs restreintes à (SAL, FON, TRI, AUT, EMP, NDE)	HAUTE	Remplacer par un code référentiel valide
employer	Détection des valeurs parasites ('AUTRE', 'LUI MEME')	HAUTE	Normaliser : mapper 'LUI MEME' vers auto-entrepreneur

3.2.4.3 Dimension Cohérence

La dimension de cohérence vérifie l'absence de contradiction logique entre plusieurs attributs interdépendants.

TABLE 11 – Dimension Cohérence — Données Tiers (Référentiel Tiers)

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
sex + civility	Genre M $\neq$ MME/MLLE ; Genre F $\neq$ M.	CRITIQUE	Résolution manuelle des incohérences de saisie
fatca_identification_number	Si nationalité ou double nationalité = US, 0 valeur nulle	CRITIQUE	Déclaration réglementaire FATCA immédiate
employer + professional_situation	Si situation = NDE (sans emploi) ou TRI (retraité), employeur doit être NULL	HAUTE	Annuler le nom d'employeur incohérent dans NOVA

3.2.4.4 Dimension Unicité

La dimension d'unicité contrôle l'absence de doublons d'enregistrements et garantit l'unicité des identifiants principaux du référentiel.

TABLE 12 – Dimension Unicité — Données Tiers (Référentiel Tiers)

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
oid / radical	Unicité stricte du numéro de radical client (0 doublon)	CRITIQUE	Fusion des doublons clients dans le référentiel
ordnum	Un seul enregistrement actif principal (main=1) par tiers	HAUTE	Éliminer les contrats de travail multiples actifs

3.2.4.5 Dimension Actualité et Pipeline

La dimension de fraîcheur et de flux s'assure du bon cadencement de l'ingestion quotidienne et de l'intégrité de la chaîne technique. Afin de s'adapter à la croissance naturelle du référentiel bancaire, le contrôle de volumétrie (**row_count**) n'est jamais figé en dur (*hardcodé*) : il compare dynamiquement le volume extrait au jour J avec la volumétrie

validée du jour précédent ($J - 1$) ainsi qu’avec le comptage de la source Oracle NOVA. Toute déviation anormale au-delà du seuil de tolérance déclenche une alerte bloquante avant de mettre à jour la nouvelle référence pour le jour suivant.

TABLE 13 – Dimension Actualité et Pipeline — Flux Batch

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
batch_datetime	Retard d’ingestion HDFS RAW < 26h	HAUTE	Alerte infrastructure Big Data et relance Sqoop
row_count	Dérive volumétrique dynamique (Source vs Cible et J vs $J - 1$) < 0.1%	CRITIQUE	Déclencher le Circuit Breaker et rejouer l’ingestion

3.3 Architecture de l’application

L’architecture présente la structure logique de la solution DQ et son déploiement physique sur l’infrastructure de production de CIH BANK.

3.3.1 Architecture logicielle (diagramme de composants)

L’architecture logicielle de notre solution s’articule autour de modules découplés. Le diagramme de la figure 9 illustre les interactions entre les modules d’ingestion, de stockage, de contrôle, de virtualisation et de restitution de la solution.

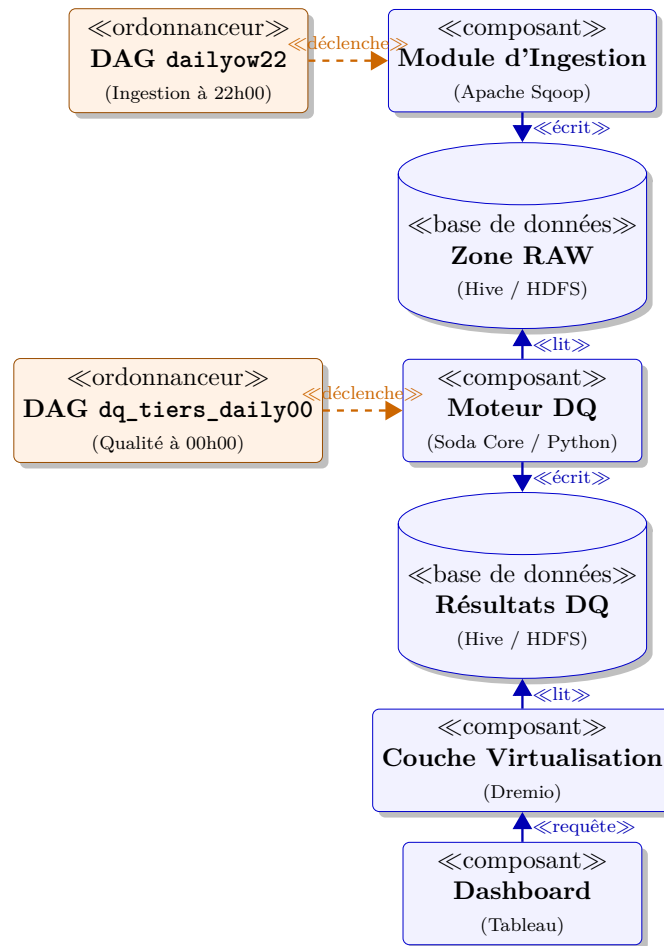


FIGURE 9 – Diagramme de composants logiciels de la solution DQ

Les composants clés de cette architecture logicielle sont détaillés ci-dessous :

3.3.1.1 Conception du moteur DQ — Soda Core

Le moteur de contrôle s'appuie sur le framework open source Soda Core, spécifiquement la bibliothèque Python `soda-core` et son connecteur Spark `soda-core-spark`. Afin d'assurer une exécution optimale dans notre écosystème Big Data, la conception logicielle repose sur les principes d'intégration suivants :

Encapsulation PySpark et Vues Temporaires : Le script d'exécution principal (`dq_tiers_checks.py`) initialise une session Spark locale. Pour chaque table de la zone RAW à contrôler (parmi la quinzaine de tables du périmètre CCM_*, telles que `ccm_npdescription` et `ccm_job` citées en exemple), la session Spark charge la table sous forme de DataFrame PySpark, puis l'enregistre en tant que vue temporaire en mémoire via la méthode `createOrReplaceTempView()`. Cela permet à Soda Core d'exécuter l'analyse de qualité directement sur ces vues mémoire sans poser de verrous ou d'accès concurrents sur les tables Hive physiques.

Définition déclarative des règles (SodaCL) : Les contrôles de qualité sont externalisés dans des fichiers de règles YAML exploitant le langage déclaratif SodaCL. Les règles fonctionnelles sont regroupées au sein d'un unique fichier métier `checks_tiers.yml`, tandis que les vérifications techniques sont isolées dans `checks_pipeline.yml`.

Extraction programmatique et typage des métadonnées : Après l'exécution du scan, le script Python exploite les structures d'API de Soda Core (l'objet `Scan`) pour parcourir la liste des vérifications évaluées. Pour chaque règle, le script extrait dynamiquement le résultat (succès/échec), la valeur de la métrique, le seuil, la dimension (Completeness, Validity, Consistency), le domaine et la criticité (définis en tant qu'attributs personnalisés dans le YAML). Un score de conformité sur 100 est attribué pour chaque règle.

Persistance partitionnée dans trois tables Hive : Après l'exécution du scan, le script consolide les résultats et écrit dans trois tables physiques Hive distinctes : `consolidated_all_column_history` pour les résultats, `consolidated_all_checks_config` pour la configuration et les requêtes SQL générées, et `consolidated_execution_log` pour la journalisation technique.

3.3.1.2 Conception de l'orchestration Airflow

La planification, le déclenchement et le monitoring des scans de qualité sont gérés par un **nouveau DAG Airflow indépendant**, nommé `dq_tiers_daily00`, spécifiquement dédié au contrôle DQ. Ce DAG ne fait pas partie des projets ETL existants ; il s'agit d'un projet d'orchestration autonome. Afin de découpler totalement l'ingestion de la vérification de la qualité, ce DAG est planifié pour s'exécuter quotidiennement à **00h00** (minuit). Cela laisse une marge de sécurité de deux heures après le déclenchement du DAG d'ingestion Sqoop principal (`dailyow22`, prévu à 22h00). L'architecture d'exécution repose sur la soumission du script de qualité autonome (notre projet DQ) directement via Airflow. Cette séparation modulaire assure qu'une défaillance dans le DAG d'ingestion ne bloque pas ou ne corrompt pas la supervision de la qualité.

3.3.1.3 Conception du dashboard Tableau

La restitution visuelle est structurée dans un dashboard Tableau interactif connecté en direct au dataset virtuel unique de Dremio. L'intelligence analytique (calcul des scores quotidiens moyens, taux de conformité globaux, regroupements par dimension de qualité ou par date) est configurée à l'aide de feuilles de calcul (sheets) et de filtres dynamiques directement dans Tableau Desktop, tirant parti de la table de faits unique. Le dashboard est structuré en trois onglets fonctionnels :

Vue d'ensemble (Dashboard Global) : Centralise les indicateurs de haut niveau (score général moyen, distribution des contrôles par statut de conformité) et présente l'évolution globale de la qualité pour les Data Owners.

Heatmap Qualité : Fournit une vue matricielle détaillée et temporelle par indicateur et par règle, permettant de repérer rapidement des dégradations soudaines ou de mesurer l'impact des actions de remédiation.

Top Anomalies : Isole les règles et les colonnes présentant des taux d'anomalies élevés, orientant ainsi les Data Stewards vers les actions correctives prioritaires dans l'application source NOVA.

Pour des raisons évidentes de sécurité et de conformité réglementaire (notamment les directives de Bank Al-Maghrib et la protection des données personnelles), toutes les données nominatives et les radicaux clients affichés dans les onglets opérationnels font

l'objet d'un masquage ou d'un floutage systématique, garantissant la stricte confidentialité des informations sensibles.

3.3.2 Architecture matérielle et diagramme de déploiement

Le diagramme de déploiement de la figure 10 modélise, conformément aux normes du standard **UML 2.5**, l'architecture physique et l'infrastructure distribuée de production sur laquelle est déployée notre solution de gouvernance et de contrôle qualité au sein de CIH BANK.

Ce modèle formalise la distribution des composants à travers trois niveaux structurants (*tiers*) : les systèmes sources et postes d'administration en amont, le cluster Big Data sécurisé au centre, et la plateforme de restitution décisionnelle en aval.

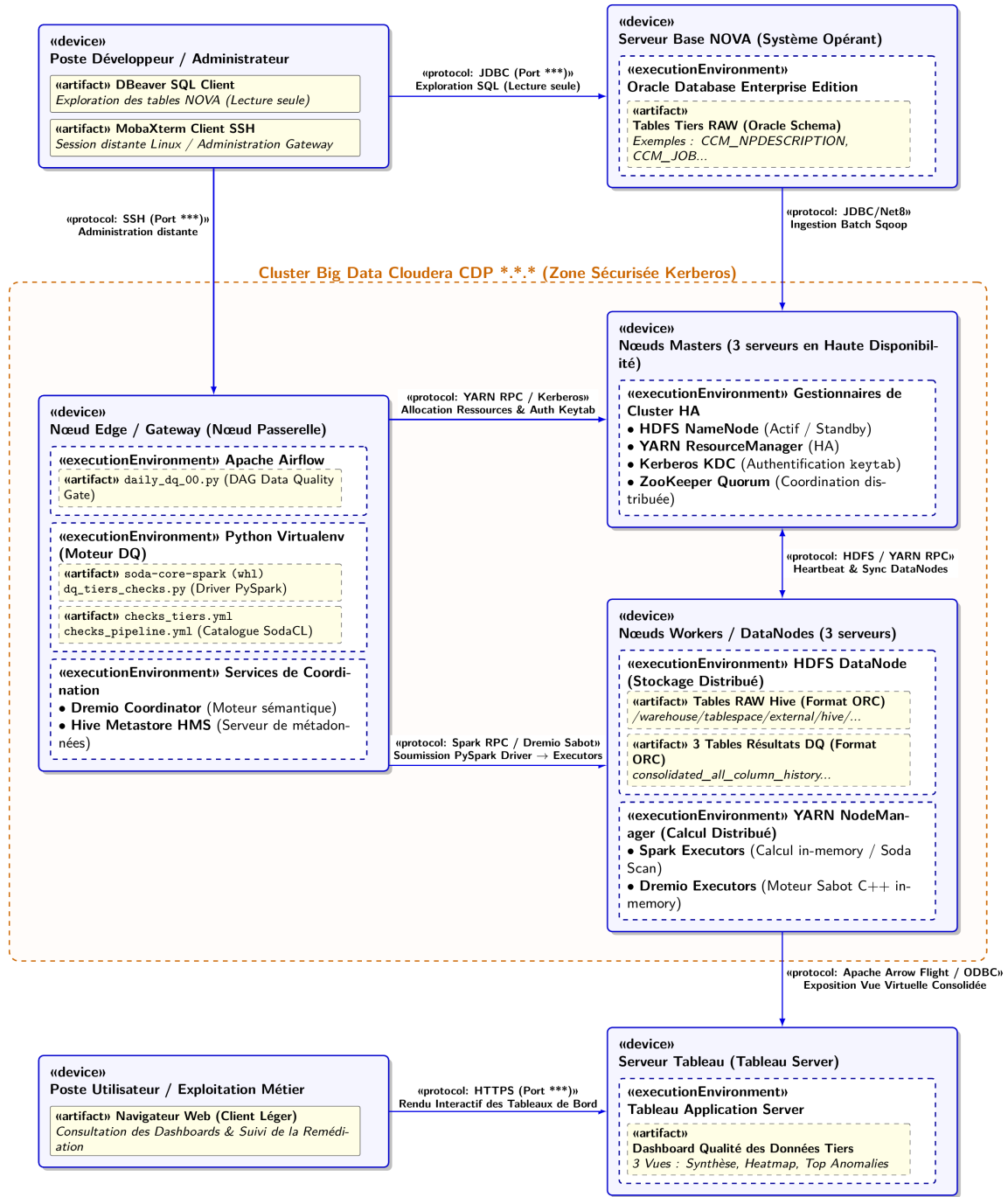


FIGURE 10 – Diagramme de déploiement UML 2.5 de la solution Data Quality

Le cluster Big Data Cloudera CDP de production de CIH BANK est structuré autour de 8 nœuds distribués, configurés de la manière suivante :

Nœud Edge (Gateway) principal : Ce serveur sert de point d'entrée pour l'administration et l'orchestration des jobs. Il héberge les workers d'Apache Airflow, le coordinateur Dremio, ainsi que le service Hive Metastore. C'est sur ce nœud qu'est installé l'environnement virtuel Python contenant les packages `soda-core` et `soda-core-spark`.

Nœuds Masters (3 serveurs en Haute Disponibilité) : Ils assurent la haute disponibilité du stockage (HDFS NameNodes configurés en Actif/Standby et JournalNodes)

et la coordination de l'ordonnancement distribué. Ils hébergent également le YARN ResourceManager pour la répartition des ressources du cluster.

Nœuds Workers (3 serveurs) : Ces serveurs gèrent le stockage local physique sous HDFS (DataNodes) et disposent des YARN NodeManagers pour allouer des conteneurs de calcul lors de l'exécution distribuée de nos requêtes PySpark et Dremio Executors.

3.3.2.1 Orchestration technique et protocole de sécurité Kerberos

Afin de sécuriser les accès aux données confidentielles du domaine Tiers de CIH BANK, l'exécution de la solution de qualité s'intègre au protocole de sécurité global du cluster :

Authentification Kerberos : Le cluster Cloudera étant sécurisé par Kerberos, l'autorisation d'accès aux répertoires HDFS RAW nécessite un ticket d'authentification valide. Avant de lancer le traitement, la tâche Airflow utilise le fichier keytab chiffré (`tiers_dq.keytab`) associé au compte de service du domaine Tiers (`tiers_dq_service`) pour générer et renouveler automatiquement le ticket Kerberos via la commande système `kinit`.

Soumission du job Spark sur YARN : Le script Python principal est exécuté en mode `spark-submit` avec le gestionnaire de ressources YARN configuré en mode `client` depuis le nœud Edge. Le driver Spark s'exécute ainsi localement sur le Gateway Node (dans le process Airflow), tandis que YARN ResourceManager alloue de manière distribuée des exécuteurs Spark au sein des nœuds Workers. Ces exécuteurs lisent et traitent en parallèle les partitions des fichiers HDFS de la zone RAW Hive.

3.3.3 Technologies de l'écosystème utilisées

Pour s'insérer au mieux au sein du système d'information décisionnel et Big Data de CIH BANK, notre solution exploite les briques technologiques de l'écosystème du Data Lake. Ces technologies sont résumées ci-dessous.



FIG. 11 – DBEaver

DBEaver — Outil d'exploration de base de données

Client SQL universel qui nous a permis d'accéder directement aux bases de données et tables du système opérant NOVA, là où sont stockées les données brutes des Tiers. Cet accès a été crucial pour explorer la structure des tables sources et définir nos règles de qualité pertinentes.



FIG. 12 – MobaXterm

MobaXterm — Client SSH et terminal distant

Outil d'administration système tout-en-un fournissant un terminal avancé pour accéder de manière sécurisée aux serveurs Linux de la banque. Il s'avère indispensable pour configurer l'environnement, déployer les scripts et tester les pipelines en ligne de commande directement sur le nœud Edge du cluster.



FIG. 13 – Apache Hive

HDFS / Apache Hive — Plateforme de stockage et de requêtage

HDFS assure le stockage persistant distribué tandis que Hive structure ces répertoires sous forme de tables SQL relationnelles. Les tables sont stockées au format **ORC** avec compression, et chaque écriture est tracée par une colonne d'audit `batch_time` conformément aux standards de l'équipe Data.



FIG. 14 – Apache Spark

Apache Spark (Spark SQL) — Moteur de traitement distribué
Framework de calcul distribué in-memory utilisé par Soda Core pour exécuter les requêtes de contrôle de qualité massives sur les données Hive avec des performances optimales.



FIG. 15 – Soda Core

Soda Core — Moteur de contrôle de la qualité (Python)
Framework de validation de données déclaratif basé sur des fichiers YAML de règles de gestion et exécuté programmatiquement sur les tables Hive RAW.



FIG. 16 – Apache Airflow

Apache Airflow 2.2.5 — Orchestrateur de tâches
Outil d'orchestration central du Data Office. Les DAGs sont structurés en tant que packages Python, construits via **PDM** en fichiers Wheel (.whl) et déployés dans le répertoire `$DAGS_FOLDER` via `pip install`. Les opérateurs utilisés incluent `BashOperator`, `PythonOperator` et `BranchPythonOperator`.



FIG. 17 – Dremio

Dremio — Moteur de virtualisation et d'exposition
Permet de virtualiser les tables de résultats Hive et d'exposer des vues SQL rapides et sécurisées, éliminant le besoin de dupliquer ou de transférer physiquement les volumes de données.



FIG. 18 – Tableau

Tableau — Outil d'analyse décisionnelle et Visualisation
Solution BI standard de CIH BANK, connectée en direct à Dremio pour fournir un reporting réactif sur la qualité des tiers à l'intention des Data Owners.



FIG. 19 – GitLab

GitLab — Gestion de versions, collaboration et suivi de projet
Plateforme DevOps centrale utilisée pour la gestion du code source (versioning Git), la revue de code (merge requests), et le suivi du backlog Data Engineering avec le Delivery Manager. GitLab centralise l'état d'avancement des tâches, assure la traçabilité des livraisons et garantit une gestion rigoureuse du cycle de vie des scripts DQ.

Conclusion

Ce chapitre a décrit la conception complète de la solution de contrôle de la qualité du domaine Tiers. Nous avons structuré la dynamique d'exécution de nos scans et le cycle de vie des anomalies identifiées. La modélisation statique a fourni la modélisation de notre table de faits, le schéma relationnel et le catalogue des règles métier de la solution. Enfin, l'architecture logicielle (composants) et matérielle (déploiement) a été détaillée au sein de l'environnement de production. Le chapitre suivant présente l'implémentation physique et les résultats opérationnels obtenus.

Chapitre 4

Réalisation de la Solution

Introduction

Ce chapitre présente la réalisation concrète du pipeline de Data Quality sur le Référentiel Tiers de CIH Bank. Tous les contrôles Soda Core s'effectuent exclusivement sur la **zone RAW** du Data Lake (base Hive `raw_nova_referentieltiers`), qui constitue la copie exacte et non transformée des tables `CCM_*` du système opérant Oracle NOVA (le périmètre complet englobe environ 15 tables Tiers ; dans ce chapitre, les tables `ccm_npdescription` et `ccm_job` serviront d'exemples illustratifs).

Nous détaillerons successivement l'environnement technique de développement et l'accès via MobaXterm, l'implémentation complète du script Soda Core avec Spark, la création des trois tables Hive physiques de persistance, l'intégration dans un DAG Airflow indépendant, la mise en place de la vue virtualisée sous Dremio, et enfin la réalisation du dashboard Tableau.

4.1 Environnement technique de développement

4.1.1 Environnement Hadoop et accès distant

Le développement et les tests ont été réalisés dans l'environnement Hadoop sécurisé de CIH Bank. L'accès aux serveurs du cluster (nœud Edge) et l'exécution des commandes en ligne de commande (CLI) s'effectuent via l'outil **MobaXterm**. MobaXterm est un terminal avancé qui fournit toutes les fonctionnalités réseaux nécessaires (SSH, SFTP) pour se connecter au cluster Big Data.

Une fois connecté au nœud Edge, l'authentification sécurisée est garantie par le protocole **Kerberos**. Conformément aux pratiques de l'équipe (identiques à celles utilisées dans les scripts Sqoop), l'accès aux tables Hive et aux fichiers HDFS s'effectue via la commande `kinit` avec un fichier keytab chiffré associé au compte de service du domaine Tiers.

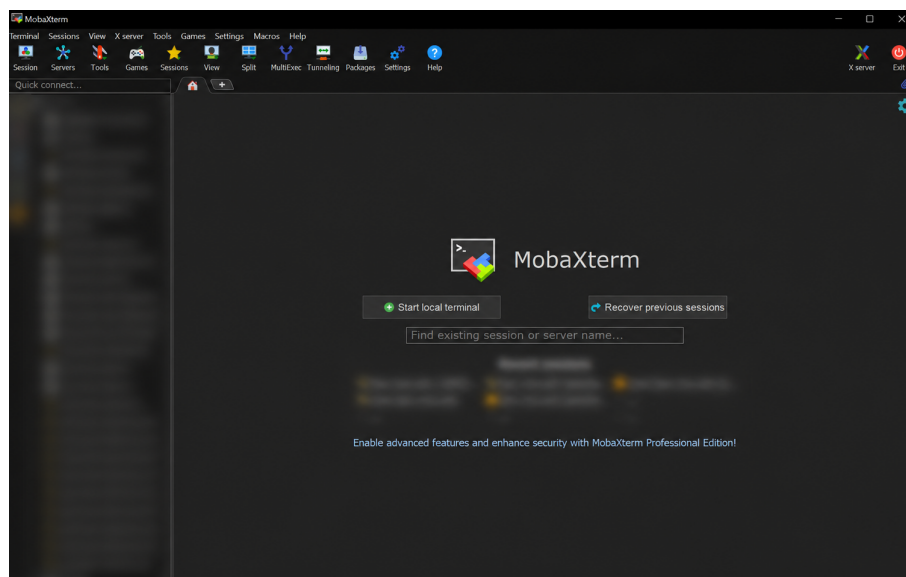


FIGURE 20 – Interface MobaXterm utilisée pour l'accès SSH au nœud Edge du cluster Cloudera

Les versions exactes des composants utilisés dans l'environnement de production sont les suivantes :

Composant	Version
Cloudera Data Platform (CDP)	Private Cloud Base 7.1.7
Apache Hadoop (HDFS / YARN)	3.1.1
Apache Hive	3.1.3
Apache Spark	3.2.0
Apache Airflow	2.5.1
Dremio	Enterprise Edition 2026
Python	3.9.13
Soda Core	soda-core-spark 3.0.45
Tableau Desktop	Professional Edition 2024.1.2

TABLE 14 – Versions des composants de l’environnement technique

4.1.2 Installation hors-ligne de Soda Core

La politique stricte de sécurité des systèmes d’information (PSSI) de CIH BANK impose une isolation réseau (air-gap) des serveurs du cluster Big Data, interdisant par conséquent tout accès direct à Internet. L’exécution standard de la commande `pip install` pour rapatrier les dépendances depuis le registre public PyPI est donc impossible. L’installation de Soda Core et de son connecteur Spark a été réalisée via une procédure d’installation hors-ligne (*Offline Wheel Installation*) articulée en trois étapes :

1. Téléchargement des archives (Wheels) : Sur un poste de travail disposant d’un accès à Internet, les archives des paquets `soda-core` et `soda-core-spark` ainsi que l’ensemble de leurs sous-dépendances ont été téléchargées au format `.whl` dans un dossier dédié via la commande `pip download`.

2. Transfert sécurisé via MobaXterm : Le dossier d’archives a été transféré de manière sécurisée en SFTP vers le nœud Edge du cluster grâce à l’interface graphique de MobaXterm.

3. Installation locale : Sur le nœud Edge, les paquets ont été déployés dans l’environnement virtuel Python (venv) alloué à l’orchestrateur Airflow à l’aide de la commande `pip`, en forçant la recherche locale :

```
1 source /opt/airflow/venv_dq/bin/activate
2 pip install --no-index --find-links=./soda_wheels soda-core-spark
```

4.2 Implémentation du moteur DQ — Soda Core et Spark

4.2.1 Arborescence et packaging du projet DQ (dq_tiers)

Le moteur de Data Quality est structuré sous la forme d’un projet Python autonome nommé `dq_tiers`, suivant scrupuleusement les standards d’ingénierie et de packaging en vigueur au sein du Data Office de CIH BANK (à l’instar des projets de transformation `py-khawarizmi` et `sddiqi`). Il est packagé via le gestionnaire moderne PDM (*Python*

Development Master) avec spécification conforme à la PEP 621 (`pyproject.toml`), et distribué sous forme de paquet **Wheel** (`.whl`).

L'arborescence du projet s'articule comme suit sur le nœud Edge :

```

1 dq_tiers/                                # Racine du projet Data
  Quality Tiers
2 |-- pyproject.toml                       # Definition des metadonnees
  et dependances PDM
3 |-- pdm.lock                             # Verrouillage deterministe
  des versions
4 |-- dq_tiers_endpoint.py                 # Point d'entree CLI principal
  du moteur DQ
5 |-- resources/
6 |   +-- configuration/
7 |     +-- checks_tiers.yml               # Catalogue declaratif des
  regles SodaCL
8 +-- src/
9   +-- dq_tiers/
10     |-- __init__.py
11     |-- context/
12     |   |-- spark_session.py            # Initialisation SparkSession
  avec support Hive
13     |   |-- pipeline_gate.py            # Module Circuit Breaker (
  fraicheur & volumetrie)
14     |   +-- scan_runner.py              # Execution distribuee des
  scans Soda Core
15     +-- persistance/
16       +-- hive_writer.py                # Module de persistance ORC en
  mode append

```

4.2.2 Principe de fonctionnement de Soda Core avec Spark

Contrairement à d'autres connecteurs SQL classiques (JDBC/ODBC) où Soda Core interroge directement la base de données, la bibliothèque `soda-core-spark-df` repose sur le traitement en mémoire distribuée. Le flux de fonctionnement s'articule en deux phases séquentielles :

Phase 1 — Contrôle de Santé du Pipeline (Data Quality Gate) : Avant toute évaluation des règles métier, le module `pipeline_gate.py` inspecte la fraîcheur des partitions HDFS brutes et la cohérence volumétrique (`row_count`). Pour éviter tout seuil statique figé (*hardcodé*) qui fausserait les alertes lors de la croissance naturelle de la base client, le calcul compare dynamiquement le volume du jour J avec la dernière référence validée de la veille ($J - 1$) et le comptage de la source Oracle NOVA. Si l'ingestion quotidienne Sqoop de 22h00 a échoué, présente un retard ($> 26h$) ou une dérive anormale ($> 0.1\%$), le *Circuit Breaker* s'active, consigne l'incident et interrompt le flux afin d'éviter la génération de faux rapports sur des données incomplètes. Dès validation du run, le nouveau `row_count` est enregistré comme référence dynamique pour le jour suivant.

Phase 2 — Évaluation Distribuée et Persistance : Une fois le pipeline validé, le script charge les tables Hive de la zone RAW en **DataFrames PySpark**, les enregistre en tant que **vues temporaires Spark** en mémoire via `createOrReplaceTempView()`, puis

traduit et soumet l'ensemble du catalogue SodaCL aux exécuteurs YARN. Les résultats sont ensuite sérialisés et persistés dans les 3 tables Hive physiques.

4.2.3 Fichier de configuration YAML (`checks_tiers.yml` — exemple)

Les règles métier sont définies de manière déclarative en utilisant le langage **SodaCL** (Soda Check Language). Ce langage permet de définir des contrôles simples (complétude, validité) ou avancés (fraîcheur, unicité, règles SQL personnalisées). Toutes les règles du domaine Tiers (KYC et CSP) sont regroupées dans un fichier unique, présenté ici en extrait illustratif sur les deux tables d'exemple `ccm_npdescription` et `ccm_job` :

```

1 # checks_tiers.yml
2 # Regles Data Quality - Domaine Tiers (KYC & CSP)
3
4 checks for ccm_npdescription
5   # --- Complétude KYC ---
6   - missing_count(name) = 0
7     name "KYC-01 | Complétude | Nom obligatoire"
8     attributes
9       criticite CRITIQUE
10      dimension completeness
11      domaine Tiers
12
13   - missing_count(first_name) = 0
14     name "KYC-02 | Complétude | Prenom obligatoire"
15     attributes
16       criticite CRITIQUE
17       dimension completeness
18
19   # --- Valider KYC ---
20   - invalid_count(gender) = 0
21     valid values ["M", "F"]
22     name "KYC-06 | Valider | Genre conforme au referentiel"
23     attributes
24       criticite HAUTE
25       dimension validity
26       domaine Tiers
27
28   # --- Unicité KYC ---
29   - duplicate_count(radical) = 0
30     name "KYC-07 | Unicité | Unicité stricte du radical client"
31     attributes
32       criticite CRITIQUE
33       dimension uniqueness
34
35 checks for ccm_job
36   # --- Complétude CSP ---
37   - missing_count(professional_sector) = 0
38     name "CSP-01 | Complétude | Secteur professionnel
39     obligatoire"
40     attributes
41       criticite CRITIQUE

```

```

41     dimension completeness
42
43     # --- Fraicheur Pipeline (Gate) ---
44     - freshness(batch_datetime) < 26h
45       name "CSP-04 | Fraicheur | Retard batch ingestion < 26h"
46       attributes
47         criticite CRITIQUE
48         dimension timeliness
49
50     # --- Regle SQL personnalisée multi-attributs (failed rows) ---
51     - failed rows
52       name "CSP-05 | Coherence | Incoherence employeur /
53       situation"
54       fail query |
55         SELECT radical, employer, professional_sector
56         FROM ccm_job
57         WHERE employer = 'SANS EMPLOI' AND professional_sector !=
58         'CHOMAGE'
59       attributes
60         criticite HAUTE
61         dimension consistency
62         domaine Tiers

```

Les fonctions d'évaluation SodaCL mobilisées dans notre catalogue de règles s'articulent ainsi :

missing_count : Dénombre les enregistrements pour lesquels l'attribut est nul ou vide, permettant de valider l'exhaustivité des champs obligatoires.

invalid_count : Évalue la conformité par rapport à un dictionnaire de valeurs autorisées (valid values) ou une expression régulière (valid regex).

duplicate_count : Identifie les doublons d'enregistrements sur les clés d'identification métier (telles que le radical client).

freshness : Mesure l'écart temporel entre la valeur maximale d'une colonne date et l'instant d'évaluation du scan, garantissant l'actualité des données.

failed rows : Permet d'injecter une requête SQL arbitraire pour vérifier des règles multi-colonnes ou des jointures complexes entre référentiels.

4.2.4 Traduction des règles SodaCL en Spark SQL

En interne, Soda Core identifie la cible (Spark) et traduit chaque règle SodaCL en une requête Spark SQL exécutée sur les vues temporaires. Voici trois exemples de cette traduction :

Exemple 1 — Complétude (missing_count) :

```

1 # >> Regle SodaCL :
2 - missing_count(name) = 0

```

⇓ Traduction automatique par Soda Core

```
1 -- >> Requete Spark SQL genereee :
2 SELECT COUNT(*) AS missing_count
3 FROM ccm_npdescription
4 WHERE name IS NULL OR TRIM(name) = ''
```

Exemple 2 — Validité (invalid_count avec referentiel) :

```
1 # >> Regle SodaCL :
2 - invalid_count(gender) = 0
3   valid values ["M", "F"]
```

⇓ Traduction automatique par Soda Core

```
1 -- >> Requete Spark SQL genereee :
2 SELECT COUNT(*) AS invalid_count
3 FROM ccm_npdescription
4 WHERE gender NOT IN ('M', 'F') OR gender IS NULL
```

Exemple 3 — Fraîcheur (freshness) :

```
1 # >> Regle SodaCL :
2 - freshness(batch_datetime) < 26h
```

⇓ Traduction automatique par Soda Core

```
1 -- >> Requete Spark SQL genereee (etape 1) :
2 SELECT MAX(batch_datetime) AS last_batch FROM ccm_job
3 -- Soda calcule ensuite le delta : CURRENT_TIMESTAMP - last_batch
4 -- et compare le resultat en heures a la limite de 26h.
```

4.2.5 Script d'exécution principal (dq_tiers_endpoint.py)

Le script Python principal orchestre l'exécution complète. Soda Core exécutant les checks sur Spark ne persiste pas automatiquement les résultats dans Hive (contrairement à l'intégration cloud). C'est notre module Python qui extrait la matrice de résultats JSON via `get_scan_results()`, l'aplatit, puis structure les données en **3 DataFrames** dont les schémas respectent exactement les tables physiques définies en conception. Les écritures se font systématiquement en mode **append** pour historiser chaque run journalier.

```
1 # dq_tiers_endpoint.py
2 import datetime, uuid, argparse
3 from datetime import date
4 from pyspark.sql import SparkSession, Row
5 from soda.scan import Scan
6
```

```

7 def run_dq_pipeline(run_date: str):
8     # --- 1. Initialisation SparkSession avec support Hive &
    Kerberos ---
9     spark = SparkSession.builder \
10         .appName("DQ_Tiers_Engine") \
11         .enableHiveSupport() \
12         .getOrCreate()
13
14     # --- 2. Chargement des tables RAW en vues temporaires Spark
    ---
15     tables_tiers = ["ccm_npdescription", "ccm_job"]
16     for table in tables_tiers:
17         spark.sql(f"SELECT * FROM raw_nova_referentieltiers.{
table}") \
18             .createOrReplaceTempView(table)
19
20     # --- 3. Execution du scan Soda Core ---
21     scan = Scan()
22     scan.set_data_source_name("spark_df")
23     scan.add_spark_session(spark)
24     scan.add_sodacl_yaml_file("resources/configuration/
checks_tiers.yml")
25     scan.execute()
26     results = scan.get_scan_results()
27
28     run_id = str(uuid.uuid4())
29     now = datetime.datetime.now()
30     history_rows, config_rows, log_rows = [], [], []
31
32     for check in results.get("checks", []):
33         qid = check.get("identity", str(uuid.uuid4()))
34         outcome = check.get("outcome", "error")
35         attrs = check.get("attributes", {})
36         dataset = check.get("dataset", "")
37         col = check.get("column", "TRANSVERSE")
38         score = 100.0 if outcome=="pass" else (50.0 if outcome
=="warn" else 0.0)
39         total = check.get("metrics", {}).get("row_count", 0)
40         or 0
41         anomalies= int(check.get("metrics", {}).get("value", 0)
42         or 0)
43
44         # Recuperation de la requete Spark SQL generee
45         sql_gen = next((q.get("sql","") for q in results.get("
queries",[]) if qid in q.get("name","")), "")
46         exec_ok = (outcome != "error")
47
48         # -- Table 1 : consolidated_all_column_history (13
    colonnes) --
49         history_rows.append(Row(
            run_id = run_id, query_id = qid, colonne_controlee =
col,
            chemin_source = f"raw_nova_referentieltiers.{dataset}"

```



```

50         table_dataset = dataset, domaine_metier = attrs.get("
domaine", "Tiers"),
51         type_regle_dq = attrs.get("dimension", "completeness"
),
52         volume_total = total, lignes_conformes = total -
anomalies,
53         score_qualite_pct = score, statut = outcome.upper(),
54         date_execution = now, criticite = attrs.get("
criticite", "HAUTE")
55     ))
56
57     # -- Table 2 : consolidated_all_checks_config (8 colonnes
) --
58     config_rows.append(Row(
59         run_id = run_id, query_id = qid, run_date = run_date,
60         table = dataset, domain = attrs.get("domaine", "Tiers
"),
61         rule = check.get("name", ""), sql = sql_gen,
62         updated_at = now, criticite = attrs.get("criticite",
"HAUTE")
63     ))
64
65     # -- Table 3 : consolidated_execution_log (10 colonnes)
--
66     log_rows.append(Row(
67         run_id = run_id, query_id = qid, run_date = run_date,
68         table = dataset, domain = attrs.get("domaine", "Tiers
"),
69         rule = check.get("name", ""),
70         check_execution_status = "SUCCESS" if exec_ok else "
ERROR",
71         check_error_message = "" if exec_ok else str(check.
get("error", "")),
72         check_timestamp = now, run_execution_status = "
SUCCESS",
73         run_error_message = "", updated_at = now
74     ))
75
76     # --- 4. Persistance en mode APPEND dans les 3 tables Hive
physiques ---
77     spark.createDataFrame(history_rows).write.mode("append") \
78         .saveAsTable("dq_results.consolidated_all_column_history
")
79     spark.createDataFrame(config_rows).write.mode("append") \
80         .saveAsTable("dq_results.consolidated_all_checks_config"
)
81     spark.createDataFrame(log_rows).write.mode("append") \
82         .saveAsTable("dq_results.consolidated_execution_log")
83
84     spark.stop()
85     print(f"[DQ SODA OK] Run {run_id} persiste avec succes ({len(
history_rows)} checks).")

```

```

86
87 if __name__ == "__main__":
88     parser = argparse.ArgumentParser()
89     parser.add_argument("--run_date", default=str(date.today()))
90     args = parser.parse_args()
91     run_dq_pipeline(args.run_date)

```

Le mode `append` est fondamental : à chaque exécution nocturne du DAG `dq_tiers_daily00`, une nouvelle tranche de résultats est ajoutée aux trois tables sans écraser les données historiques. Cette accumulation constitue la base temporelle qui alimente les tendances et analyses dans le dashboard Tableau.

4.3 Création des tables Hive physiques de persistance

Les trois tables de persistance, définies lors de la conception, sont créées physiquement dans la base Hive `dq_results`. Ces tables stockent l'intégralité des métriques : les scores de qualité, les définitions SQL générées, et les statuts d'exécution. L'utilisation du format ORC avec compression Snappy garantit de très hautes performances lors des requêtes analytiques ultérieures.

```

1 CREATE DATABASE IF NOT EXISTS dq_results;
2
3 -- 1. Table Historique des scores et resultats par colonne
4 CREATE TABLE IF NOT EXISTS dq_results.
5     consolidated_all_column_history (
6         run_id                STRING,
7         query_id              STRING,
8         colonne_controlee     STRING,
9         chemin_source         STRING,
10        table_dataset         STRING,
11        domaine_metier        STRING,
12        type_regle_dq         STRING,
13        volume_total          BIGINT,
14        lignes_conformes      BIGINT,
15        score_qualite_pct     DOUBLE,
16        statut                STRING,
17        date_execution        TIMESTAMP,
18        criticite              STRING
19    ) STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');
20
21 -- 2. Table Configuration et referentiel des regles (SQL genere)
22 CREATE TABLE IF NOT EXISTS dq_results.
23     consolidated_all_checks_config (
24         run_id                STRING,
25         query_id              STRING,
26         run_date              STRING,
27         table                 STRING,
28         domain                STRING,
29         rule                  STRING,
30         sql                   STRING,
31         updated_at            TIMESTAMP,
32         criticite              STRING

```

```

31 ) STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');
32
33 -- 3. Table Journal d'exécution et logs techniques
34 CREATE TABLE IF NOT EXISTS dq_results.consolidated_execution_log
35 (
36     run_id                STRING,
37     query_id              STRING,
38     run_date              STRING,
39     table                 STRING,
40     domain                STRING,
41     rule                  STRING,
42     check_execution_status STRING,
43     check_error_message   STRING,
44     check_timestamp       TIMESTAMP,
45     run_execution_status  STRING,
46     run_error_message     STRING,
47     updated_at            TIMESTAMP
48 ) STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

```

4.4 Création du DAG Airflow indépendant (dq_tiers_daily00)

Pour garantir un découplage total tout en sécurisant la chaîne de traitement, un nouveau DAG autonome nommé `dq_tiers_daily00` a été configuré dans l'orchestrateur Apache Airflow de CIH BANK.

Ce DAG est planifié quotidiennement à **00h00** (minuit), laissant deux heures de marge de sécurité après le lancement du DAG d'ingestion Sqoop (`dailyow22`, planifié à 22h00). Il implémente le pattern de **Data Quality Gate (Circuit Breaker)** découpé en trois tâches séquentielles :

1. Tâche `check_pipeline_gate` : Exécute en amont la vérification de fraîcheur (< 26h) et de cohérence volumétrique sur les tables RAW HDFS. Si le batch d'ingestion a échoué ou a pris du retard, le DAG lève une exception bloquante et s'arrête immédiatement, empêchant le calcul de faux indicateurs sur des données périmées.

2. Tâche `run_business_dq` : Déclenche le script principal du moteur DQ (`dq_tiers_endpoint.py`) dans l'environnement virtuel Python dédié (`venv_dq`) pour évaluer l'ensemble des règles métier SodaCL sur le cluster via Spark et persister les résultats dans Hive.

```

1 # /opt/airflow/dags/dq_tiers_daily00.py
2 from airflow import DAG
3 from airflow.operators.bash import BashOperator
4 from datetime import datetime, timedelta
5
6 default_args = {
7     'owner': 'data_office',
8     'depends_on_past': False,
9     'start_date': datetime(2026, 2, 9),
10    'email_on_failure': True,
11    'email': ['dataengineers@cihbank.ma'],
12    'retries': 1,

```

```

13     'retry_delay': timedelta(minutes=5)
14 }
15
16 with DAG(
17     dag_id='dq_tiers_daily00',
18     default_args=default_args,
19     description='Pipeline automatisé de Data Quality Tiers (KYC &
20     CSP)',
21     schedule_interval='0 0 * * *',
22     catchup=False,
23     tags=['DQ', 'Tiers', 'SodaCore', 'Spark']
24 ) as dag:
25
26     # 1. Gate : Verification de fraicheur et integrite
27     volumetrique du flux RAW
28     check_pipeline_gate = BashOperator(
29         task_id='check_pipeline_gate',
30         bash_command=(
31             'source /opt/airflow/venv_dq/bin/activate && '
32             'python -m dq_tiers.context.pipeline_gate --domain
33             Tiers --run_date {{ ds }}'
34         )
35     )
36
37     # 2. Execution distribuee du moteur de qualite Soda Core sur
38     Spark & persistance
39     run_business_dq = BashOperator(
40         task_id='run_business_dq',
41         bash_command=(
42             'source /opt/airflow/venv_dq/bin/activate && '
43             'kinit -kt /etc/security/keytabs/dq_svc.keytab
44             dq_svc@CIH.MA && '
45             'python /opt/airflow/projects/dq_tiers/
46             dq_tiers_endpoint.py --run_date {{ ds }}'
47         )
48     )
49
50     # Ordonnancement s quentiel avec Circuit Breaker
51     check_pipeline_gate >> run_business_dq

```

4.5 Exposition et virtualisation sous Dremio

Une fois les trois tables physiques alimentées dans Hive (`consolidated_all_column_history`, `consolidated_all_checks_config`, et `consolidated_execution_log`), il est nécessaire de consolider ces informations pour les exposer à l'outil BI Tableau de manière claire et optimale.

Plutôt que de créer une table physique supplémentaire ou d'effectuer des jointures complexes et coûteuses dans Tableau, nous utilisons **Dremio** en tant que couche de virtualisation sémantique.

Dans Dremio, nous créons un dataset virtuel (VDS) nommé `fact_tiers_dq_consolidated`. Cette vue réalise dynamiquement la jointure sur les clés uniques `run_id` et `query_id` des

trois tables sources de Hive. Le résultat est une table de faits consolidée, exposant la totalité de la vision 360° des règles DQ, sans aucune duplication physique des données. Dremio accélère les temps de réponse de cette jointure virtuelle grâce à ses réflexions matérielles (Data Reflections).

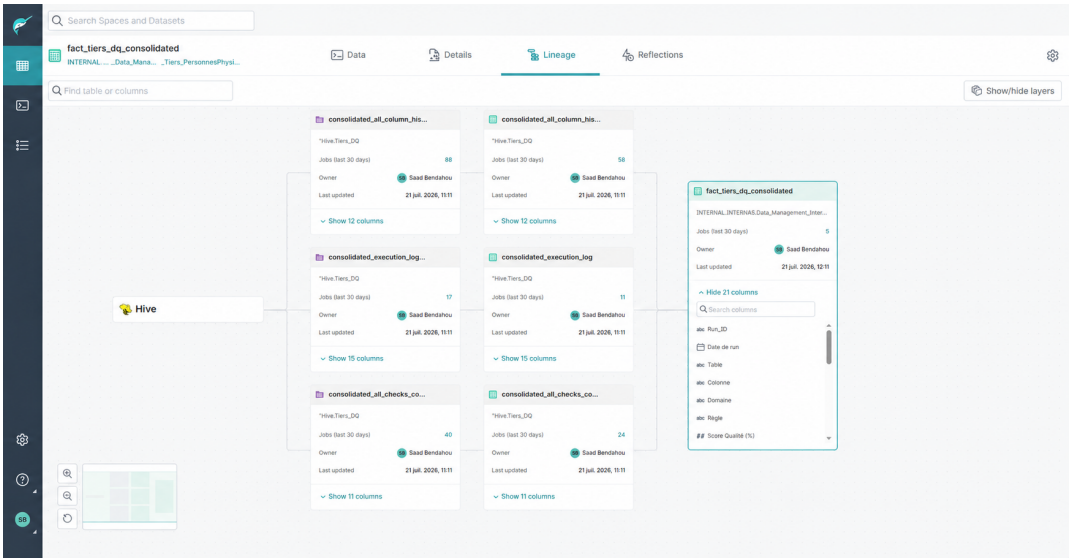


FIGURE 21 – Lignage des données dans Dremio montrant la création de la vue sémantique à partir des 3 tables Hive

4.6 Réalisation du dashboard Tableau

4.6.1 Connexion Tableau à Dremio et traitement analytique

Le dashboard Tableau est connecté à Dremio via le connecteur ODBC/JDBC natif. L'unique vue virtuelle `fact_tiers_dq_consolidated` est importée comme source de données unique dans Tableau Desktop.

Tous les filtres (par date d'exécution, par sous-domaine, par dimension de qualité), ainsi que toutes les agrégations (comptage de règles, calculs des moyennes de taux de conformité quotidiens ou scores moyens) sont réalisés de manière dynamique au sein des feuilles de calcul (sheets) et des tableaux de bord de Tableau en tirant directement les champs de la table de faits.

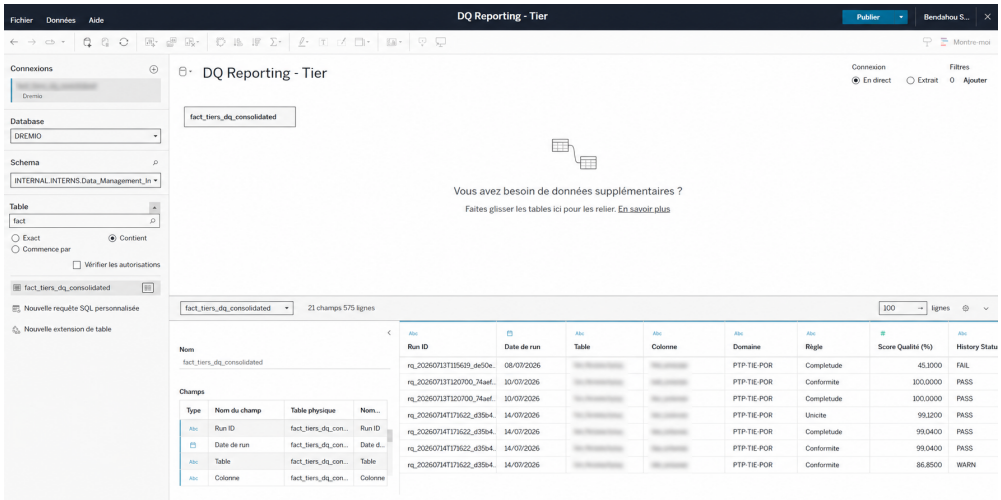


FIGURE 22 – Interface de connexion entre Tableau Desktop et le serveur Dremio

Note de confidentialité : Il est important de souligner que, pour des raisons évidentes de sécurité et de respect du secret bancaire, les noms de tables réels et certaines données sensibles présentées dans les captures d’écran suivantes ont fait l’objet d’un masquage ou d’un floutage systématique.

4.6.2 Onglet 1 — Vue d’ensemble (Dashboard Global)

La figure 23 présente la page d’accueil du dashboard, offrant une vision synthétique de l’état de santé du référentiel.

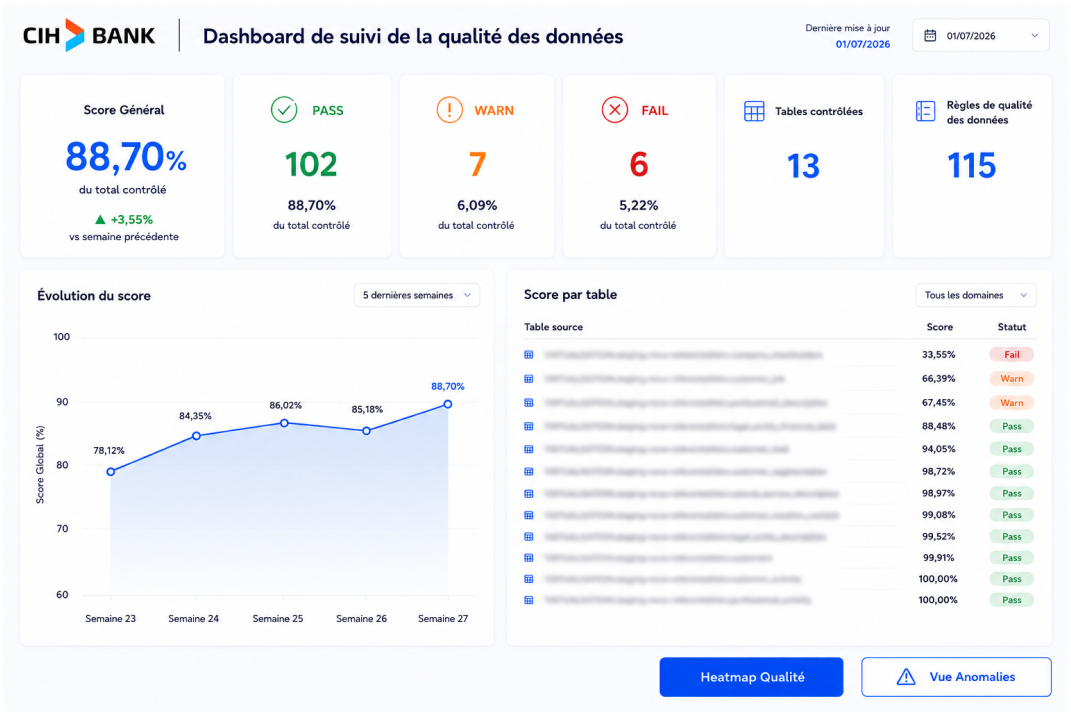


FIGURE 23 – Dashboard Tableau — Vue d’ensemble de la qualité des données

Cet onglet centralise les indicateurs de pilotage de haut niveau nécessaires aux décideurs et aux Data Owners :

Score Général de qualité : Affiche l’indicateur global (ex : 88,70%) ainsi que sa variation comparative par rapport à la semaine précédente.

Répartition des contrôles par statut : Dénombre et quantifie les tests validés (PASS), en avertissement (WARN) ou en échec critique (FAIL) avec leurs pourcentages respectifs.

Périmètre et couverture : Précise le volume de tables sous contrôle et le nombre total de règles de qualité déployées en production.

Évolution temporelle du score : Retracer la dynamique de qualité sur les 5 dernières semaines sous forme de courbe d’analyse de tendance.

Score par table : Classement interactif permettant d’identifier immédiatement les tables sources nécessitant des actions d’assainissement prioritaires.

4.6.3 Onglet 2 — Heatmap Qualité

La figure 24 présente la vue matricielle détaillée.



FIGURE 24 – Dashboard Tableau — Heatmap Qualité par colonne et par date

Cet onglet permet une analyse granulaire et temporelle du niveau de qualité :

Granularité par indicateur : Liste exhaustivement chaque attribut sous contrôle (birthdate, gender, city, etc.) associé à sa règle et sa dimension DQ.

Matrice thermique temporelle (Heatmap) : Colore les résultats quotidiens selon un gradient allant du rouge (non-conformité) au vert (conformité totale). Cette représentation visuelle permet de détecter instantanément les incidents techniques ponctuels ou de valider l’impact positif des campagnes de remédiation.

4.6.4 Onglet 3 — Top Anomalies

La figure 25 présente la vue orientée sur l’analyse et la remédiation.

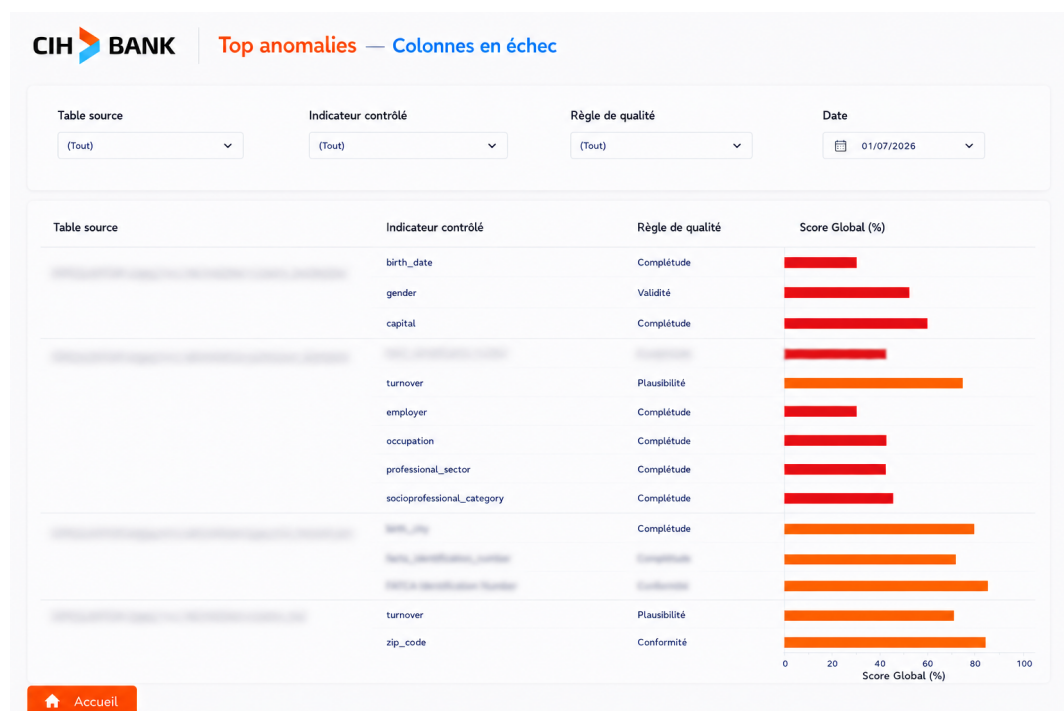


FIGURE 25 – Dashboard Tableau — Top anomalies et colonnes en échec

Ce dernier onglet constitue l’outil d’action opérationnelle pour les Data Stewards :

Ciblage des défaillances critiques : Isole les règles et attributs présentant les taux d’anomalies les plus sévères.

Diagramme de distribution des scores : Un graphique en barres horizontales met en exergue le niveau de sévérité des défaillances pour chaque indicateur.

Filtrage dynamique contextuel : Des filtres interactifs (par table, par attribut, par dimension ou par date) permettent d’isoler rapidement les causes racines pour engager les corrections requises dans le système source NOVA.

Conclusion

Ce chapitre a présenté l’implémentation concrète des différents composants du pipeline de Data Quality Tiers : le script Soda Core avec ses fichiers de configuration YAML, les tables Hive de persistance, la tâche Airflow post-ingestion, l’exposition sémantique de Dremio et le dashboard Tableau de visualisation. L’ensemble de ces composants forme un pipeline automatisé, opérationnel chaque nuit à 00h00, qui permet de mesurer, historiser et restituer la qualité des données métier du Référentiel Tiers.

Conclusion Générale et Perspectives

Conclusion générale

Au terme de ce projet de fin d'études, nous avons conçu, développé et industrialisé un pipeline automatisé de contrôle de la qualité des données (Data Quality) intégré à l'écosystème Big Data de CIH BANK. Ce travail répond aux problématiques rencontrées par l'institution bancaire dans la gestion de volumes de données toujours plus importants, où les traitements manuels et les opérations de remédiation curatives deviennent rapidement coûteux, chronophages et sources d'erreurs ou de non-conformité réglementaire.

L'étude préalable du contexte métier a permis d'identifier plusieurs limites des méthodes traditionnelles de gestion de la donnée, notamment l'absence de validation à la source, la difficulté d'obtenir une vision claire de l'état de santé du Data Lake, ainsi que le manque de traçabilité des anomalies liées aux informations d'identification KYC et aux catégories socio-professionnelles (CSP). Afin de répondre à ces contraintes et de se conformer aux exigences de la réglementation (loi 09-08 de la CNDP, RGPD, BCBS 239), une solution complète reposant sur des technologies modernes a été proposée et déployée.

Le pipeline développé met en œuvre une architecture robuste permettant de prendre en charge l'ensemble du cycle de fiabilisation de la donnée. Celui-ci comprend l'ingestion automatisée depuis le système source via Apache Sqoop, l'exécution distribuée de règles de qualité complexes (complétude, validité, cohérence croisée) grâce à l'intégration du framework Soda Core au sein de jobs Apache Spark, le stockage des résultats dans la zone HDFS (format ORC), ainsi que la modélisation sémantique à travers le moteur de virtualisation Dremio. L'intégration de ces résultats dans un tableau de bord interactif développé sur Tableau Software constitue également une contribution majeure, offrant aux Data Stewards un outil de supervision clair et précis.

Le projet ne s'est pas limité au développement des fonctionnalités analytiques. Une attention particulière a également été portée à l'orchestration et à l'industrialisation de la solution. La chaîne complète a été planifiée et surveillée via Apache Airflow, permettant d'automatiser le déclenchement des contrôles quotidiennement tout en s'intégrant parfaitement avec les dépendances du système d'information de la banque. Cette démarche contribue à réduire les interventions manuelles, à assurer une observabilité totale du flux de données et à garantir la fiabilité de la solution en production.

Les différents tests réalisés tout au long du projet ont permis de valider le bon fonctionnement des modules développés et leur impact positif sur l'écosystème analytique de la banque. Les objectifs fixés au début du projet ont ainsi été atteints, tout en proposant une architecture modulaire, évolutive et facilement maintenable. Au-delà des résultats obtenus, ce projet nous a permis de mettre en pratique les connaissances acquises au cours de notre formation d'ingénieur dans des domaines variés tels que le développement logiciel, l'architecture Big Data, le Data Engineering, les bases de données distribuées, la Business Intelligence ainsi que la méthodologie d'industrialisation logicielle. Cette expérience constitue une étape particulièrement enrichissante sur les plans technique et

méthodologique, représentant une véritable immersion dans les exigences d'un projet industriel bancaire.

Perspectives

Bien que les résultats obtenus soient très satisfaisants, trois perspectives d'évolution majeures peuvent être envisagées afin d'enrichir davantage la solution développée et de pousser plus loin la valorisation des données :

Élargissement du périmètre fonctionnel : Actuellement centré sur le domaine des Tiers (données KYC et CSP), le pipeline a vocation à être étendu à d'autres domaines cruciaux du système d'information de CIH BANK. Il s'agirait d'intégrer les données issues du *Core Banking*, notamment les écritures comptables, la comptabilité générale, ainsi que l'ensemble de la partie finance. Cette généralisation permettrait de créer une plateforme centralisée garantissant la fiabilité des données à l'échelle de toute la banque.

Remédiation intelligente via une approche LLM local : Pour pallier les limites des règles purement déterministes, l'intégration de modèles de langage (LLM) exécutés localement constitue une piste prometteuse. Cette approche permettrait de remédier automatiquement à certains problèmes de qualité de données minimales ou de valeurs manquantes (par exemple, inférer contextuellement un genre à partir d'un prénom). L'exécution locale de ces modèles au sein de l'infrastructure de la banque est primordiale pour garantir la stricte confidentialité et la souveraineté des informations.

Intégration d'un système d'alerte multicanal intelligent : Afin de rendre le dispositif encore plus proactif, il serait pertinent de le coupler à un système de notification multicanal (via e-mail, Microsoft Teams, etc.). Ce système serait capable d'alerter automatiquement et de manière ciblée les équipes métiers en cas de dégradation soudaine du score de qualité d'une table ou d'anomalies critiques détectées par Airflow, favorisant ainsi une réactivité immédiate sans nécessiter de surveillance manuelle continue.

Ces évolutions permettraient de faire mûrir progressivement l'infrastructure actuelle vers un système autonome et proactif, véritable centre névralgique du pilotage de la santé des données au sein de l'institution.

Bibliographie et Webographie

Bibliographie

- [1] DAMA International, *DAMA-DMBOK : Data Management Body of Knowledge*, 2nd ed. Basking Ridge, NJ, USA : Technics Publications, 2017.

Webographie

- [1] <https://innowise.com/blog/data-analytics-in-banking/> : How data analytics is transforming the banking and financial services industry - Innowise.
- [2] https://scielo.org.za/scielo.php?script=sci_arttext&pid=S2304-82632024000200007 : Artificial intelligence technology to enhance data quality management practices in the banking industry in South Africa.
- [3] <https://www.ewsolutions.com/financial-data-quality-management/> : Financial Data Quality Management : Unlocking the Power for Competitive Advantage.
- [4] <https://www.ibm.com/new/product-blog/bcbs239-compliance> : BCBS 239 compliance : Key findings, failures and fixes with watsonx.data intelligence - IBM.
- [5] https://www.researchgate.net/publication/389700967_Assessing_Data_Quality_Challenges_on_International_Banking_In_Case_of_Commercial_Bank_of_Ethiopia : (PDF) Assessing Data Quality Challenges on International Banking : In Case of Commercial Bank of Ethiopia - ResearchGate.
- [6] <https://corp-im.com/cost-of-bad-data-decisions/> : Cost of Bad Data Decisions : Hidden Financial Damage (2026) - Corpim.
- [7] https://www.researchgate.net/publication/282829737_Data_quality_in_banking_regulatory_requirements_and_best_practices : Data quality in banking : Regulatory requirements and best practices | Request PDF.
- [8] <https://sbs-software.com/insights/artificial-intelligence-data/data-driven-transformation-banking/> : Data-driven transformation in European banking - SBS Software.
- [9] <https://www.actian.com/blog/data-management/the-costly-consequences-of-poor-data-quality/> : The Consequences of Poor Data Quality : Uncovering the Hidden Risks - Actian Corporation.
- [10] <https://open.metu.edu.tr/handle/11511/25576> : Data quality assessment in credit risk management by customized total data quality management approach - Open-METU.
- [11] <https://etd.lib.metu.edu.tr/upload/12619894/index.pdf> : data quality assessment in credit risk management by a.

- [12] <https://www.collibra.com/blog/the-6-dimensions-of-data-quality> : The 6 Data Quality Dimensions with Examples - Collibra.
- [13] <https://www.integrate.io/blog/data-quality-improvement-stats-from-etl/> : Data Quality Improvement Stats from ETL – 50+ Key Facts Every Data Leader Should Know in 2026 | Integrate.io.
- [14] <https://eminence.ch/en/cost-poor-data-quality/> : The \$12.9M Drain : The Real Cost of Poor Data Quality - agence Eminence.
- [15] <https://www.doubletrack.com/post/hidden-cost-dirty-data> : The Hidden Cost of Dirty Data : \$617B, Per Our 2026 Data Report.
- [16] <https://veridion.com/blog-posts/banking-capital-markets-data-strategy-development/> : How to Develop a Data Strategy in Banking and Capital Markets - Veridion.
- [17] <https://www.mdpi.com/1424-8220/25/14/4345> : Quantifying the Multidimensional Impact of Cyber Attacks in Digital Financial Services : A Systematic Literature Review - MDPI.
- [18] <https://www.bis.org/publ/bcbs239.pdf> : Basel Committee on Banking Supervision – Principles for effective risk data aggregation and risk reporting - Bank for International Settlements.
- [19] <https://montecarlo.ai/blog-bcbs-239-data-quality/> : BCBS 239 Data Quality Requirements : How Banks Can Ensure Compliance.
- [20] <https://www.solidatus.com/bcbs-239/> : What is BCBS 239 ? A Summary of Key Principles & Compliance - Solidatus.
- [21] <https://www.fluxforce.ai/regulations/bcbs-bcbs-239-risk-data-aggregation/> : BCBS 239 : Requirements, Scope, Penalties & Compliance - FluxForce AI.
- [22] <https://www.collibra.com/blog/how-to-achieve-data-quality-excellence-for-bcbs-239-risk-data-aggregation-compliance> : BCBS 239 data quality excellence for risk compliance - Collibra.
- [23] https://www.ey.com/en_nl/industries/banking-capital-markets/why-bcbs-239-compliance-is-essential-in-2025 : Why BCBS 239 Compliance is essential in 2025 | EY - Netherlands.
- [24] <https://soda.io/use-cases/bcbs239-compliance> : BCBS 239 Compliance with Soda Data Quality.
- [25] https://iason-onigiri-prod.s3.eu-south-1.amazonaws.com/Credit_Risk_Basel_IV_Regulatory_Framework_and_New_Frontiers_8db63aced7.pdf : Credit Risk : Basel IV Regulatory Framework and New Frontiers - AWS.
- [26] <https://kpmg.com/xx/en/our-insights/ecb-office/ecb-ratchets-up-the-pressure-on-risk-data-aggregation.html> : ECB ratchets up the pressure on risk data aggregation - KPMG International.
- [27] <https://www.oliverwyman.com/our-expertise/insights/2024/jul/data-management-risk-reporting-challenges-eu-banks-bcbs239.html> : Data Management And Risk Reporting Challenges For EU Banks - Oliver Wyman.
- [28] <https://www.grantthornton.com.cy/insights/quantitative-risk-articles/ECB-Guide-on-effective-risk-data-aggregation-and-risk-reporting/> : ECB Guide on effective risk data aggregation and risk reporting - Grant Thornton Cyprus.

- [29] <https://validio.io/blog/bcbs-239-from-regulatory-burden-to-data-quality> : BCBS 239 : Turning regulatory burden into data quality excellence - Validio.
- [30] <https://www.bairdholm.com/blog/citigroup-fined-136-million-totaling-536-million-since-2020/> : Citigroup Fined \$136 Million, Totaling \$536 Million Since 2020 | Baird Holm LLP.
- [31] <https://www.bankingsupervision.europa.eu/press/pr/date/2026/html/ssm.pr260219~93d5b6f73e.sv.html> : ECB sanctions J.P. Morgan for misreporting capital requirements - Banking supervision.
- [32] <https://basecapanalytics.com/the-real-cost-of-bad-data/> : Citigroup fined for bad loan data - Basecap Analytics.
- [33] <https://www.banque-france.fr/en/press-release/ecb-sanctions-bofa-securities-europe-sa-breaching-reporting-requirements> : ECB sanctions BofA Securities Europe SA for breaching reporting requirements.
- [34] <https://www.federalreserve.gov/newsevents/pressreleases/enforcement20240710a.htm> : Federal Reserve Board fines Citigroup \$60.6 million for violating the Board's 2020 enforcement action.
- [35] <https://www.experian.co.uk/blogs/latest-thinking/data-quality/hidden-cost-of-untrusted-data/> : The hidden cost of untrusted data at AI scale - Experian UK.
- [36] <https://store.ectap.ro/articole/699.pdf> : Credit Risk Assessment under Basel Accords - Theoretical and Applied Economics.
- [37] <https://www.ibm.com/think/insights/cost-of-poor-data-quality> : The True Cost of Poor Data Quality - IBM.
- [38] <https://www.querysurge.com/resource-center/white-papers/the-data-validation-deficit-analyzing-banking-pain-points-and-the-quest-for-effective-solutions> : White Papers - Analyzing Banking Pain Points and the Quest for Effective Solutions.
- [39] <https://journals.co.za/doi/abs/10.7553/90-2-2392> : Artificial intelligence technology to enhance data quality management practices in the banking industry in South Africa - Sabinet African Journals.
- [40] https://ideas.repec.org/p/bdi/opques/qef_666_22.html : A decision-making rule to detect insufficient data quality : an application of statistical learning techniques to the non-performing loans banking data? - IDEAS/RePEc.
- [41] <https://barrmoses.medium.com/citigroup-fined-136m-for-bad-data-what-can-we-learn-1f15f82ea928> : Citigroup Fined \$136M for Bad Data. What Can We Learn? | by Barr Moses - Medium.
- [42] <https://docs.soda.io/soda-core/overview-main.html> : Soda Core Official Documentation - Data Quality Management.
- [43] <https://spark.apache.org/docs/latest/> : Apache Spark Official Documentation - Unified Engine for Large-Scale Data Analytics.
- [44] <https://airflow.apache.org/docs/> : Apache Airflow Official Documentation - A platform to programmatically author, schedule, and monitor workflows.
- [45] <https://docs.dremio.com/> : Dremio Documentation - The Easy and Open Data Lakehouse.

- [46] <https://docs.python.org/3/> : Python Software Foundation - Official Documentation.
- [47] <https://docs.cloudera.com/> : Cloudera Documentation - Enterprise Data Cloud.